

Amazon Product Recommendation System

Hybrid Recommendation Engine (Content-Based + Collaborative Filtering)

Problem Statement

Design and evaluate a hybrid recommendation system for an e-commerce platform using product metadata and user behavior to deliver personalized, relevant, and diverse product recommendations.

Approach Overview

This project implements three recommendation strategies:

- **Content-Based Filtering** Uses product metadata (name, description, category, price, discount) with TF-IDF vectorization to recommend similar products.
 - **Collaborative Filtering (User-User)** Leverages user-item interactions and cosine similarity to recommend products based on similar users' behavior.
 - **Hybrid Model (Final Solution)** Combines both approaches using score fusion: **Final Score = $0.6 \times \text{CF} + 0.4 \times \text{Content} + 0.1 \times \text{Popularity}$**
-

Key Techniques

- Text vectorization using **TF-IDF**
 - Similarity computation using **cosine similarity**
 - Feature engineering (price difference, value-for-money score, weighted rating)
 - Handling sparsity using fallback strategies
 - Cold-start solutions for new users and products
-

Evaluation Strategy

Models were evaluated using:

- **Precision@5** → recommendation accuracy
- **Coverage** → variety of products recommended
- **Diversity** → category spread in recommendations

Due to extreme sparsity ($\approx 0.09\%$ interaction density), Precision@5 is low, highlighting the importance of hybrid methods.

Key Findings

- Collaborative Filtering struggles due to sparse user interactions
 - Content-Based Filtering improves diversity and handles cold-start scenarios
 - **Hybrid Model performs best**, balancing relevance, diversity, and coverage
 - Popularity boosting improves recommendation reliability in sparse cases
-

Business Impact

- Improved product discovery through personalized recommendations
 - Increased engagement and conversion using hybrid strategy
 - Robust handling of cold-start users and products
 - Scalable design suitable for real-world deployment
-

Deployment Strategy

- API: FastAPI
 - Containerization: Docker
 - Orchestration: Kubernetes (AWS EKS)
 - Caching: Redis
 - Scheduling: Airflow
 - Monitoring: Prometheus + **Evidently AI / PSI for model drift**
-

Key KPIs

- Click-Through Rate (CTR)
 - Conversion Rate
 - Revenue per User (RPU)
 - Retention Rate
 - Coverage & Diversity
-

Conclusion

The hybrid recommendation system effectively combines user behavior and product metadata to deliver robust, scalable, and high-quality recommendations, making it suitable for production-level e-commerce applications.

Problem statement: Build and evaluate a hybrid recommendation engine (content-based+ collaborative filtering) for an e-commerce platform using product metadata and user behavior data.

Section A:

Data Understanding & Cleaning: Understand the dataset like a data scientist in a product analytics team. Summarize the dataset: Number of unique users, products, and reviews
Top 5 categories by number of products Price range and discount insights Clean and preprocess the data: Convert prices to numeric Parse categories into hierarchy levels
Normalize rating scores and count outliers or Create derived features like price_difference, value_for_money_score, weighted ratings. Handle missing values or anomalies Remove duplicates, invalid records Handle missing ratings/reviews with appropriate strategy

Section B:

Exploratory Data Analysis: Think like a product analyst trying to identify buying patterns
Visualize: Most reviewed products Visualize top 10 categories by number of products.
Average rating per category Discounts vs actual price correlation User Engagement
Insights (5 marks) ○ Which products have high ratings but low review counts? ○ Are highly rated products also heavily reviewed? Create 3 actionable insights for Amazon's product strategy based on EDA.

Section C:

Content-Based Filtering: Act like a content engineer personalizing user feeds based on product metadata. Vectorize product text (about_product + product_name) using: TF-IDF or embeddings Build a product similarity matrix Recommend top 5 similar products to: A new product with no reviews A product with high user dropout (bad ratings) Add category, price, and discount to enhance content vectors Evaluate recommendations: How diverse and relevant are the content-based results?

Section D:

Collaborative Filtering (User-Item) : Now you're a machine learning engineer building smart recommendations using user behavior. Create a user-item matrix using user_id, product_id, and rating. Apply: User-User Collaborative Filtering (cosine similarity) OR Item-Item Collaborative Filtering (cosine or Pearson) Recommend top 5 unseen products per user

Section E:

Hybrid Recommender (Content + Collaborative) : Step into the role of a senior ML engineer combining models for better performance. Design a hybrid strategy: Score fusion: $0.6 * CF_score + 0.4 * Content_score$ Compare recommendation quality of hybrid vs individual methods. Evaluate hybrid system on: A cold-start product (new product) A cold-start user (few reviews) Suggest how to improve hybrid performance further using real-world constraints like: Popularity Recent purchases Product availability

Section F:

Bonus: Business Strategy & Deployment Which model works best for new users ? Which model works best for returning users ? How can we recommend products with no ratings ? How would you deploy this system in production? Mention tools/technologies.

What KPIs should Amazon track to measure success?

◆ Section : Data Load

```
In [1]: !pip install gdown
import gdown

# — Standard Library —
import os
import zipfile

# — Data —
import numpy as np
import pandas as pd

# — Visualisation —
import matplotlib.pyplot as plt
import seaborn as sns

# — ML —
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
```

```
Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages (5.2.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from gdown) (3.25.2)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2026.2.25)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (1.7.1)
```

Download file

```
In [2]: def download_from_drive(file_id: str, output_file: str) -> str:
        """
        Download a file from Google Drive using file ID.

        Args:
            file_id (str): Google Drive file ID
            output_file (str): Output file name (with extension)

        Returns:
            str: Path of downloaded file
        """
        url = f"https://drive.google.com/uc?id={file_id}"

        gdown.download(url, output_file, quiet=False)

        print("Download Complete!")
        return output_file
```

```
In [3]: file_id = "10A0wOG1epBxHAMNXun7kwNr_QsupUMzj"
        output = "amazon_dataset.zip"

        download_from_drive(file_id, output)
```

```
Downloading...
From: https://drive.google.com/uc?id=10A0wOG1epBxHAMNXun7kwNr_QsupUMzj
To: /content/amazon_dataset.zip
100%|██████████| 2.04M/2.04M [00:00<00:00, 55.5MB/s]
Download Complete!
```

```
Out[3]: 'amazon_dataset.zip'
```

UNZIP FILE

```
In [4]: def extract_zip(zip_file_path: str, extract_to: str = None) -> str:
        """
        Extract a ZIP file to a specified directory.

        Args:
            zip_file_path (str): Path to the ZIP file
            extract_to (str, optional): Directory to extract files into.
                                     Defaults to folder with same name as ZIP.

        Returns:
            str: Path where files were extracted
        """
        if not os.path.exists(zip_file_path):
            raise FileNotFoundError(f"{zip_file_path} not found")

        # Default folder name = zip file name without extension
        if extract_to is None:
            extract_to = os.path.splitext(zip_file_path)[0]

        os.makedirs(extract_to, exist_ok=True)

        with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
            zip_ref.extractall(extract_to)

        print("Files extracted successfully!")
        return extract_to
```

```
In [5]: zip_file_name = "/content/amazon_dataset.zip"

        extract_zip(zip_file_name, "AmazonDataset")
```

Files extracted successfully!

```
Out[5]: 'AmazonDataset'
```

Load CSV

```
In [6]: def load_csv(file_path: str, show_head: bool = True, n_rows: int = 5) -> pd.DataFrame:
        """
        Load a CSV file into a pandas DataFrame.

        Args:
            file_path (str): Path to the CSV file
            show_head (bool): Whether to print the first few rows
            n_rows (int): Number of rows to display

        Returns:
            pd.DataFrame: Loaded DataFrame
        """
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"{file_path} not found")

        df = pd.read_csv(file_path)

        print("CSV loaded successfully!")

        if show_head:
            #print(df.head(n_rows))
            print(df.head())
```

```
return df
```

```
In [7]: csv_file = "/content/AmazonDataset/amazon.csv"

df = load_csv(csv_file)
```

CSV loaded successfully!

	product_id	product_name		category	discounted_price	
0	B07JW9H4J1	Wayona Nylon Braided USB to Lightning Fast Cha...		Computers&Accessories Accessories&Peripherals ...	₹399	
1	B098NS6PVG	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...		Computers&Accessories Accessories&Peripherals ...	₹199	
2	B096MSW6CT	Sounce Fast Phone Charging Cable & Data Sync U...		Computers&Accessories Accessories&Peripherals ...	₹199	
3	B08HDJ86NZ	boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...		Computers&Accessories Accessories&Peripherals ...	₹329	
4	B08CF3B7N1	Portronics Konnect L 1.2M Fast Charging 3A 8 P...		Computers&Accessories Accessories&Peripherals ...	₹154	

	actual_price	discount_percentage	rating	rating_count	
0	₹1,099	64%	4.2	24,269	
1	₹349	43%	4.0	43,994	
2	₹1,899	90%	3.9	7,928	
3	₹699	53%	4.2	94,363	
4	₹399	61%	4.2	16,905	

	about_product	
0	High Compatibility : Compatible With iPhone 12...	
1	Compatible with all Type C enabled devices, be...	
2	【 Fast Charger& Data Sync】-With built-in safet...	
3	The boAt Deuce USB 300 2 in 1 cable is compati...	
4	[CHARGE & SYNC FUNCTION]- This cable comes wit...	

	user_id	
0	AG3D604STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB...	
1	AECPPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX...	
2	AGU3BBQ2V2DDAMOAKGFAWDDQ6QHA,AESFLDV2PT363T2AQ...	
3	AEWAZDZZJLQUYVOVGBEUKSLXHQA,AG5HTSFRRE6NL3M5S...	
4	AE3Q6KSUK5P75D5HFYHCRAOLODSA,AFUGIFH5ZAFXRDSZH...	

	user_name	
0	Manav,Adarsh gupta,Sundeep,S.Sayeed Ahmed,jasp...	
1	ArdKn,Nirbhay kumar,Sagar Viswanathan,Asp,Plac...	
2	Kunal,Himanshu,viswanath,sai niharka,saqib mal...	
3	Omkar dhale,JD,HEMALATHA,Ajwadh a.,amar singh ...	
4	rahuls6099,Swasat Borah,Ajay Wadke,Pranali,RVK...	

	review_id	
0	R3HXWT0LRP0NMF,R2AJM3LFTLZHFO,R6AQJGUP6P86,R1K...	
1	RGIQEG07R9HS2,R1SMWZQ86XIN8U,R2J3Y1WL29GWDE,RY...	
2	R3J3EEQ9TZI5ZJ,R3E7WBGK7ID0KV,RWU79XKQ6I1QF,R2...	
3	R3EEUZKKK9J36I,R3HJVYCLYOY554,REDECAZ7AMPQC,R1...	
4	R1BP4L2HH9TFUP,R16PVJEXKV6QZS,R2UPDB81N66T4P,R...	

	review_title	
0	Satisfied,Charging is really fast,Value for mo...	
1	A Good Braided Cable for Your Type C Device,Go...	
2	Good speed for earlier versions,Good Product,W...	
3	Good product,Good one,Nice,Really nice product...	
4	As good as original,Decent,Good one for second...	

	review_content	
0	Looks durable Charging is fine tooNo complains...	
1	I ordered this cable to connect my phone to An...	

```

2 Not quite durable and sturdy,https://m.media-a...
3 Good product,long wire,Charges good,Nice,I bou...
4 Bought this instead of original apple, does th...

```

```

                                img_link \
0 https://m.media-amazon.com/images/W/WEBP_40237...
1 https://m.media-amazon.com/images/W/WEBP_40237...
2 https://m.media-amazon.com/images/W/WEBP_40237...
3 https://m.media-amazon.com/images/I/41V5FtEWPk...
4 https://m.media-amazon.com/images/W/WEBP_40237...

```

```

                                product_link
0 https://www.amazon.in/Wayona-Braided-WN3LG1-Sy...
1 https://www.amazon.in/Ambrane-Unbreakable-Char...
2 https://www.amazon.in/Sounce-iPhone-Charging-C...
3 https://www.amazon.in/Deuce-300-Resistant-Tang...
4 https://www.amazon.in/Portronics-Konnect-POR-1...

```

Section A: Data Understanding & Cleaning:

```

In [8]: def inspect_dataframe(df, show_head: bool = True, n_rows: int = 5):
        """
        Display basic information about a DataFrame.

        Args:
            df (pd.DataFrame): Input DataFrame
            show_head (bool): Whether to show top rows
            n_rows (int): Number of rows to display
        """
        print("Shape of DataFrame:")
        print(df.shape)

        print("\nDataFrame Info:")
        print(df.info())

        if show_head:
            print("\nTop rows:")
            print(df.head(n_rows))

```

```

In [9]: inspect_dataframe(df)

```

Shape of DataFrame:
(1465, 16)

DataFrame Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1465 entries, 0 to 1464

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	product_id	1465 non-null	object
1	product_name	1465 non-null	object
2	category	1465 non-null	object
3	discounted_price	1465 non-null	object
4	actual_price	1465 non-null	object
5	discount_percentage	1465 non-null	object
6	rating	1465 non-null	object
7	rating_count	1463 non-null	object
8	about_product	1465 non-null	object
9	user_id	1465 non-null	object
10	user_name	1465 non-null	object
11	review_id	1465 non-null	object
12	review_title	1465 non-null	object
13	review_content	1465 non-null	object
14	img_link	1465 non-null	object
15	product_link	1465 non-null	object

dtypes: object(16)

memory usage: 183.3+ KB

None

Top rows:

	product_id	product_name
0	B07JW9H4J1	Wayona Nylon Braided USB to Lightning Fast Cha...
1	B098NS6PVG	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...
2	B096MSW6CT	Sounce Fast Phone Charging Cable & Data Sync U...
3	B08HDJ86NZ	boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...
4	B08CF3B7N1	Portronics Konnect L 1.2M Fast Charging 3A 8 P...

	category	discounted_price
0	Computers&Accessories Accessories&Peripherals ...	₹399
1	Computers&Accessories Accessories&Peripherals ...	₹199
2	Computers&Accessories Accessories&Peripherals ...	₹199
3	Computers&Accessories Accessories&Peripherals ...	₹329
4	Computers&Accessories Accessories&Peripherals ...	₹154

	actual_price	discount_percentage	rating	rating_count
0	₹1,099	64%	4.2	24,269
1	₹349	43%	4.0	43,994
2	₹1,899	90%	3.9	7,928
3	₹699	53%	4.2	94,363
4	₹399	61%	4.2	16,905

	about_product
0	High Compatibility : Compatible With iPhone 12...
1	Compatible with all Type C enabled devices, be...
2	【 Fast Charger& Data Sync】-With built-in safet...
3	The boAt Deuce USB 300 2 in 1 cable is compati...
4	[CHARGE & SYNC FUNCTION]- This cable comes wit...

	user_id
0	AG3D604STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB...

```

1 AECPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX...
2 AGU3BBQ2V2DDAMOAKGFAWDDQ6QHA,AESFLDV2PT363T2AQ...
3 AEWAZDZZJLQUYVOVGBEUKSLXHQ5A,AG5HTSFRRE6NL3M5S...
4 AE3Q6KSUK5P75D5HFYHCRAOLODSA,AFUGIFH5ZAFXRDSZH...

```

```

                                user_name \
0 Manav,Adarsh gupta,Sundeep,S.Sayeed Ahmed,jasp...
1 ArdKn,Nirbhay kumar,Sagar Viswanathan,Asp,Plac...
2 Kunal,Himanshu,viswanath,sai niharka,saqib mal...
3 Omkar dhale,JD,HEMALATHA,Ajwadh a.,amar singh ...
4 rahuls6099,Swasat Borah,Ajay Wadke,Pranali,RVK...

```

```

                                review_id \
0 R3HXWT0LRP0NMF,R2AJM3LFTLZHFO,R6AQJGUP6P86,R1K...
1 RGIQEG07R9HS2,R1SMWZQ86XIN8U,R2J3Y1WL29GWDE,RY...
2 R3J3EQQ9TZI5ZJ,R3E7WBGK7ID0KV,RWU79XKQ6I1QF,R2...
3 R3EEUZKKK9J36I,R3HJVYCLYOY554,REDECAZ7AMPQC,R1...
4 R1BP4L2HH9TFUP,R16PVJEXKV6QZS,R2UPDB81N66T4P,R...

```

```

                                review_title \
0 Satisfied,Charging is really fast,Value for mo...
1 A Good Braided Cable for Your Type C Device,Go...
2 Good speed for earlier versions,Good Product,W...
3 Good product,Good one,Nice,Really nice product...
4 As good as original,Decent,Good one for second...

```

```

                                review_content \
0 Looks durable Charging is fine tooNo complains...
1 I ordered this cable to connect my phone to An...
2 Not quite durable and sturdy,https://m.media-a...
3 Good product,long wire,Charges good,Nice,I bou...
4 Bought this instead of original apple, does th...

```

```

                                img_link \
0 https://m.media-amazon.com/images/W/WEBP_40237...
1 https://m.media-amazon.com/images/W/WEBP_40237...
2 https://m.media-amazon.com/images/W/WEBP_40237...
3 https://m.media-amazon.com/images/I/41V5FtEWPk...
4 https://m.media-amazon.com/images/W/WEBP_40237...

```

```

                                product_link
0 https://www.amazon.in/Wayona-Braided-WN3LG1-Sy...
1 https://www.amazon.in/Ambrane-Unbreakable-Char...
2 https://www.amazon.in/Sounce-iPhone-Charging-C...
3 https://www.amazon.in/Deuce-300-Resistant-Tang...
4 https://www.amazon.in/Portronics-Konnect-POR-1...

```

• Number of unique users, products, and reviews

Why This Matters

High users + low reviews → sparse data

High reviews per product → strong collaborative filtering

Few reviews per user → cold-start risk

```
In [10]: def get_dataset_stats(df, user_col='user_id', product_col='product_id', review_c
        """
```

```
Calculate basic dataset statistics.
```

```
Args:
```

```
df (pd.DataFrame): Input DataFrame  
user_col (str): Column name for users  
product_col (str): Column name for products  
review_col (str): Column name for reviews
```

```
Returns:
```

```
dict: Dictionary with counts
```

```
"""
```

```
num_users = df[user_col].nunique()  
num_products = df[product_col].nunique()  
num_reviews = df[review_col].nunique()
```

```
print("Unique Users:", num_users)  
print("Unique Products:", num_products)  
print("Total Reviews:", num_reviews)
```

```
return {  
    "unique_users": num_users,  
    "unique_products": num_products,  
    "total_reviews": num_reviews  
}
```

```
In [11]: stats = get_dataset_stats(df)
```

```
Unique Users: 1194  
Unique Products: 1351  
Total Reviews: 1194
```

Interaction Density

Business Insight

If:

Most users review only 1 product → sparse matrix

Few products have most reviews → popularity bias

Reviews Per User & Reviews Per Product

```
In [12]: def review_distribution_stats(df, user_col='user_id', product_col='product_id',
```

```
        """
```

```
Analyze review distribution per user and per product.
```

```
Args:
```

```
df (pd.DataFrame): Input DataFrame  
user_col (str): Column name for users  
product_col (str): Column name for products  
review_col (str): Column name for reviews
```

```
Returns:
```

```
dict: Summary statistics for users and products
```

```
"""
```

```
# Reviews per user
```

```
reviews_per_user = df.groupby(user_col)[review_col].count()
```

```

user_stats = reviews_per_user.describe()

print("Reviews per User Stats:")
print(user_stats)

# Reviews per product
reviews_per_product = df.groupby(product_col)[review_col].count()
product_stats = reviews_per_product.describe()

print("\nReviews per Product Stats:")
print(product_stats)

return {
    "user_distribution": user_stats,
    "product_distribution": product_stats
}

```

In [13]: stats = review_distribution_stats(df)

```

Reviews per User Stats:
count      1194.000000
mean         1.226968
std           0.772262
min           1.000000
25%           1.000000
50%           1.000000
75%           1.000000
max           10.000000
Name: review_id, dtype: float64

```

```

Reviews per Product Stats:
count      1351.000000
mean         1.084382
std           0.331529
min           1.000000
25%           1.000000
50%           1.000000
75%           1.000000
max           3.000000
Name: review_id, dtype: float64

```

Basic Cleaning and remove duplicates

Convert Prices to Numeric

Convert Discount %

Clean Rating Columns

Parse Category Hierarchy

```

In [14]: def clean_price(val):
        if pd.isna(val):
            return np.nan
        val = str(val).replace('₹', '').replace(',', '').strip()
        try:
            return float(val)

```

```
except:
    return np.nan
```

```
In [15]: def clean_amazon_dataset(df: pd.DataFrame):
        """
        Clean and preprocess Amazon dataset.
        Returns:
            df (cleaned DataFrame),
            unique_categories (array)
        """
        before = len(df)

        # --- Remove duplicates ---
        df = df.drop_duplicates()

        # --- Remove missing critical fields ---
        df = df.dropna(subset=['product_id', 'user_id'])

        # --- Clean price columns ---
        df['actual_price'] = df['actual_price'].apply(clean_price)
        df['discounted_price'] = df['discounted_price'].apply(clean_price)

        # --- Discount ---

        df['discount_percentage'] = (df['discount_percentage']
                                     .astype(str).str.replace('%', '').str.strip()
                                     .apply(pd.to_numeric, errors='coerce'))

        print(df[['actual_price', 'discounted_price', 'discount_percentage']].dtypes)

        # --- Ratings ---
        df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
        df['rating_count'] = pd.to_numeric(df['rating_count'], errors='coerce')
        df['rating_count'] = df['rating_count'].fillna(0)

        df = df[df['rating'].between(1, 5)]

        # Remove rows with invalid ratings (must be between 1 and 5)
        df = df[(df['rating'] >= 1) & (df['rating'] <= 5)]

        after = len(df)
        print(f"Row Removed {before - after} rows ({before} → {after})")

        df = df[df['rating'].between(1, 5)]

        # --- Category processing ---
        df['category'] = df['category'].astype(str)

        df['category_depth'] = df['category'].apply(lambda x: len(x.split('|')))
        max_levels = df['category_depth'].max()

        cat_cols = [f'cat_lvl{i+1}' for i in range(max_levels)]
        df[cat_cols] = df['category'].str.split('|', expand=True)

        for col in cat_cols:
            df[col] = df[col].str.strip()
            df[col] = df[col].str.replace('&', ' & ', regex=False)
```

```
# --- Unique categories ---
unique_categories = (
    df['category']
    .str.split('|')
    .explode()
    .str.strip()
    .unique()
)

# --- Debug info ---
print("Final Shape:", df.shape)
print("Max Category Depth:", max_levels)
print("Total Unique Categories:", len(unique_categories))

return df, unique_categories
```

```
In [16]: df, unique_categories = clean_amazon_dataset(df)

print(unique_categories)
```

```

actual_price          float64
discounted_price      float64
discount_percentage   int64
dtype: object
Row Removed 1 rows (1465 → 1464)
Final Shape: (1464, 24)
Max Category Depth: 7
Total Unique Categories: 317
['Computers&Accessories' 'Accessories&Peripherals' 'Cables&Accessories'
 'Cables' 'USBCables' 'NetworkingDevices' 'NetworkAdapters'
 'WirelessUSBAdapters' 'Electronics' 'HomeTheater,TV&Video' 'Accessories'
 'HDMICables' 'Televisions' 'SmartTelevisions' 'RemoteControls'
 'StandardTelevisions' 'TVMounts,Stands&Turntables' 'TVWall&CeilingMounts'
 'RCACables' 'HomeAudio' 'SpeakerAccessories' 'Mounts' 'OpticalCables'
 'Projectors' 'Adapters' 'SatelliteEquipment' 'SatelliteReceivers'
 'DVICables' 'SpeakerCables' 'MediaStreamingDevices' 'StreamingClients'
 'AVReceivers&Amplifiers' 'Speakers' 'TowerSpeakers' '3DGlasses'
 'WearableTechnology' 'SmartWatches' 'Mobiles&Accessories'
 'MobileAccessories' 'Chargers' 'PowerBanks' 'Smartphones&BasicMobiles'
 'Smartphones' 'MemoryCards' 'MicroSD' 'BasicMobiles'
 'Headphones,Earbuds&Accessories' 'Headphones' 'In-Ear'
 'AutomobileChargers' 'AutomobileAccessories' 'Cradles' 'WallChargers'
 'Cables&Adapters' 'OTGAdapters' 'Photo&VideoAccessories' 'Tripods'
 'SelfieSticks' 'Stands' 'CableConnectionProtectors' 'Décor'
 'Maintenance,Upkeep&Repairs' 'ScreenProtectors' 'StylusPens'
 'Bedstand&DeskMounts' 'Cases&Covers' 'BasicCases' 'HandlebarMounts'
 'On-Ear' 'LaptopAccessories' 'CameraPrivacyCovers' 'PhoneCharms'
 'Shower&WallMounts' 'ExternalDevices&DataStorage' 'PenDrives'
 'Keyboards,Mice&InputDevices' 'Mice' 'GraphicTablets' 'Lapdesks'
 'NotebookComputerStands' 'Keyboards' 'MusicalInstruments' 'Microphones'
 'Condenser' 'GeneralPurposeBatteries&BatteryChargers'
 'DisposableBatteries' 'OfficeProducts' 'OfficePaperProducts' 'Paper'
 'Stationery' 'Pens,Pencils&WritingSupplies' 'Pens&Refills'
 'GelInkRollerballPens' 'Home&Kitchen' 'CraftMaterials' 'Scrapbooking'
 'Tape' 'Keyboard&MouseSets' 'ExternalHardDisks' 'Cameras&Photography'
 'VideoCameras' 'Tripods&Monopods' 'Tabletop&TravelTripods'
 'OfficeElectronics' 'Calculators' 'Scientific' 'Repeaters&Extenders'
 'TripodLegs' 'Printers,Inks&Accessories' 'Inks,Toners&Cartridges'
 'InkjetInkCartridges' 'Keyboard&MiceAccessories' 'DustCovers'
 'PCGamingPeripherals' 'GamingMice' 'PaintingMaterials' 'Paints'
 'MousePads' 'HardDiskBags' 'Flashes' 'Macro&RinglightFlashes' 'Routers'
 'Over-Ear' 'BluetoothSpeakers' 'Notebooks,WritingPads&Diaries'
 'WireboundNotebooks' 'RechargeableBatteries' 'BluetoothAdapters'
 'USBtoUSBAdapters' 'CompleteTripodUnits' 'Notepads&MemoBooks' 'Film'
 'Monitors' 'USBGadgets' 'Lamps' 'Cleaners' 'CleaningKits'
 'SecurityCameras' 'DomeCameras' 'TabletAccessories' 'Gamepads' 'Basic'
 'USBHubs' 'Audio&VideoAccessories' 'PCMicrophones' 'OutdoorSpeakers'
 'Bags&Sleeves' 'LaptopSleeves&Slipcases' 'ExternalMemoryCardReaders'
 'BottledInk' 'CompositionNotebooks' 'RetractableBallpointPens'
 'EthernetCables' 'Components' 'Memory' 'UninterruptedPowerSupplies'
 'Cases' 'SecureDigitalCards' 'Flashes&SelfieLights' 'SelfieLights'
 'Webcams&VoIPEquipment' 'Webcams' 'CoolingPads' 'HomeImprovement'
 'Electrical' 'Adapters&Multi-Outlets' 'Copy&PrintingPaper'
 'ColouredPaper' 'InternalSolidStateDrives' 'MultimediaSpeakerSystems'
 'DataCards&Dongles' 'LaptopChargers&PowerSupplies' 'PCSpeakers'
 'Batteries&Chargers' 'BatteryChargers' 'Bags,Cases&Sleeves'
 'StickBallpointPens' 'DrawingMaterials' 'DrawingMedia' 'Pencils'
 'WoodenPencils' 'InternalHardDrives' 'Printers' 'Pens' 'SATACables'
 'PCHeadsets' 'GamingKeyboards' 'SoundbarSpeakers' 'Earpads'
 'InkjetPrinters' 'Toys&Games' 'Arts&Crafts' 'Drawing&PaintingSupplies'

```

```

'ColouringPens&Markers' 'Headsets' 'ExternalSolidStateDrives'
'PowerLANAdapters' 'InkjetInkRefills&Kits' 'PhotoStudio&Lighting'
'PhotoBackgroundAccessories' 'BackgroundSupports' 'Financial&Business'
'PowerAccessories' 'SurgeProtectors' 'Tablets' 'CordManagement'
'TonerCartridges' 'LiquidInkRollerballPens' 'FountainPens'
'HardDriveAccessories' 'Caddies' 'Laptops' 'TraditionalLaptops'
'Kitchen&HomeAppliances' 'SmallKitchenAppliances'
'Kettles&HotWaterDispensers' 'ElectricKettles'
'Heating,Cooling&AirQuality' 'RoomHeaters' 'ElectricHeaters' 'FanHeaters'
'Vacuum,Cleaning&Ironing' 'Irons,Steamers&Accessories' 'LintShavers'
'DigitalKitchenScales' 'Kitchen&Dining' 'KitchenTools'
'ManualChoppers&Chippers' 'Choppers' 'InductionCooktop' 'HandBlenders'
'Irons' 'DryIrons' 'MixerGrinders' 'WaterHeaters&Geysers'
'InstantWaterHeaters' 'Kettle&ToasterSets' 'StorageWaterHeaters'
'ImmersionRods' 'DeepFatFryers' 'AirFryers' 'HomeStorage&Organization'
'LaundryOrganization' 'LaundryBaskets' 'SteamIrons' 'JuicerMixerGrinders'
'Vacuums&FloorCare' 'Vacuums' 'HandheldVacuums' 'EggBoilers'
'SandwichMakers' 'MiniFoodProcessors&Choppers' 'DigitalScales'
'VacuumSealers' 'Fans' 'CeilingFans' 'CanisterVacuums'
'PressureWashers,Steam&WindowCleaners' 'HalogenHeaters' 'Pop-upToasters'
'HeatConvectors' 'Coffee,Tea&Espresso' 'CoffeeGrinders'
'ElectricGrinders' 'ExhaustFans' 'DripCoffeeMachines'
'WaterPurifiers&Accessories' 'WaterPurifierAccessories' 'WaterCartridges'
'Rice&PastaCookers' 'Car&Motorbike' 'CarAccessories'
'InteriorAccessories' 'AirPurifiers&Ionizers' 'Wet-DryVacuums'
'AirPurifiers' 'HEPAAirPurifiers' 'WaterFilters&Purifiers' 'LaundryBags'
'SewingMachines&Accessories' 'Sewing&EmbroideryMachines'
'IroningAccessories' 'SprayBottles' 'HandMixers' 'Mills&Grinders'
'WetGrinders' 'OvenToasterGrills' 'Juicers' 'Health&PersonalCare'
'HomeMedicalSupplies&Equipment' 'HealthMonitors' 'WeighingScales'
'DigitalBathroomScales' 'EspressoMachines' 'TableFans' 'MilkFrothers'
'Humidifiers' 'SmallApplianceParts&Accessories' 'StandMixerAccessories'
'RoboticVacuums' 'YogurtMakers' 'ColdPressJuicers' 'AirConditioners'
'Split-SystemAirConditioners' 'WaffleMakers&Irons' 'StovetopEspressoPots'
'CoffeeMakerAccessories' 'MeasuringSpoons' 'CoffeePresses' 'RotiMakers'
'Parts&Accessories' 'FanParts&Accessories' 'StandMixers' 'PedestalFans'
'VacuumAccessories' 'VacuumBags' 'HandheldBags']

```

Top 5 Categories Visualization

```

In [17]: def plot_top_categories(df, category_col='cat_lv11', product_col='product_id', t
        """
        Function to display top N categories by number of unique products
        and plot them as a bar chart.

        Parameters:
            df (DataFrame): Input dataframe
            category_col (str): Category column name
            product_col (str): Product ID column name
            top_n (int): Number of top categories to display
        """

        # Get top categories
        top_categories = (
            df.groupby(category_col)[product_col]
              .nunique()
              .sort_values(ascending=False)
              .head(top_n)
        )

```

```

print(f"Top {top_n} Categories by Product Count:")
print(top_categories.to_string())

# Plot
plt.figure(figsize=(10, 5))
bars = plt.bar(top_categories.index, top_categories.values,
               color='steelblue', edgecolor='black')

plt.title(f"Top {top_n} Categories by Number of Products", fontsize=14)
plt.xlabel("Category")
plt.ylabel("Number of Unique Products")
plt.xticks(rotation=30, ha='right')

#top_categories.plot(kind='barh', figsize=(8,4), title='Top 5 categories by

# Add value Labels
for bar in bars:
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.5,
             str(int(bar.get_height())),
             ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.show()

```

In [18]: `# — Top 5 Categories by Number of Products —————`
`plot_top_categories(df, category_col='category', product_col='product_id', top_n`

Top 5 Categories by Product Count:

category

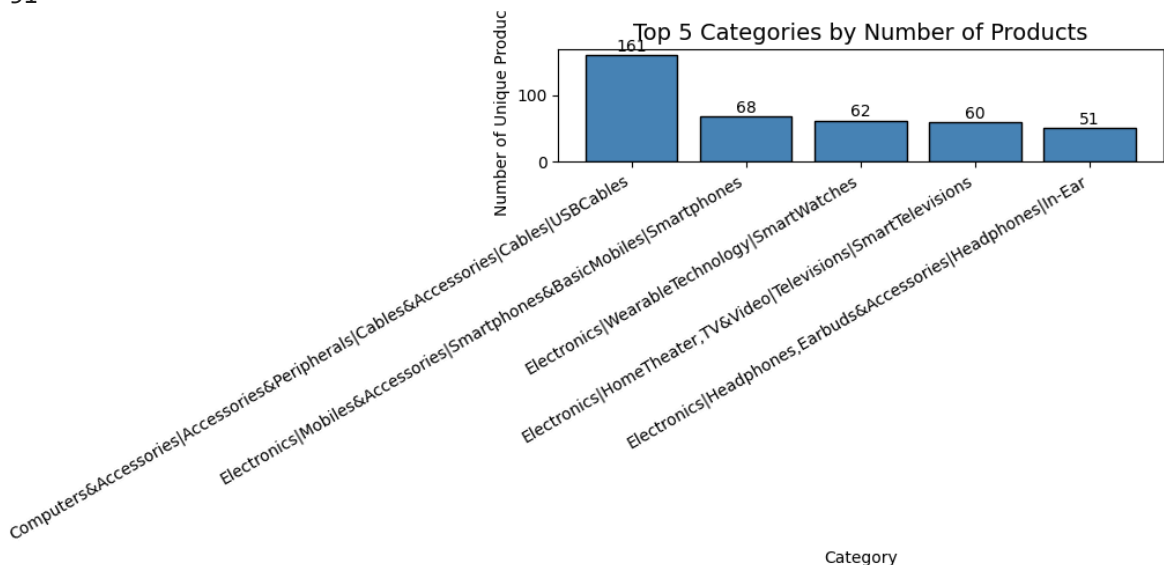
Computers&Accessories|Accessories&Peripherals|Cables&Accessories|Cables|USBCables
161

Electronics|Mobiles&Accessories|Smartphones&BasicMobiles|Smartphones
68

Electronics|WearableTechnology|SmartWatches
62

Electronics|HomeTheater,TV&Video|Televisions|SmartTelevisions
60

Electronics|Headphones,Earbuds&Accessories|Headphones|In-Ear
51



In [19]: `def plot_top_categories(df, category_col='category', product_col='product_id', t`
`"""`

Plots top N categories based on unique product count.

Parameters:

df (pd.DataFrame): Input dataframe
category_col (str): Column name for category
product_col (str): Column name for product ID
top_n (int): Number of top categories to display

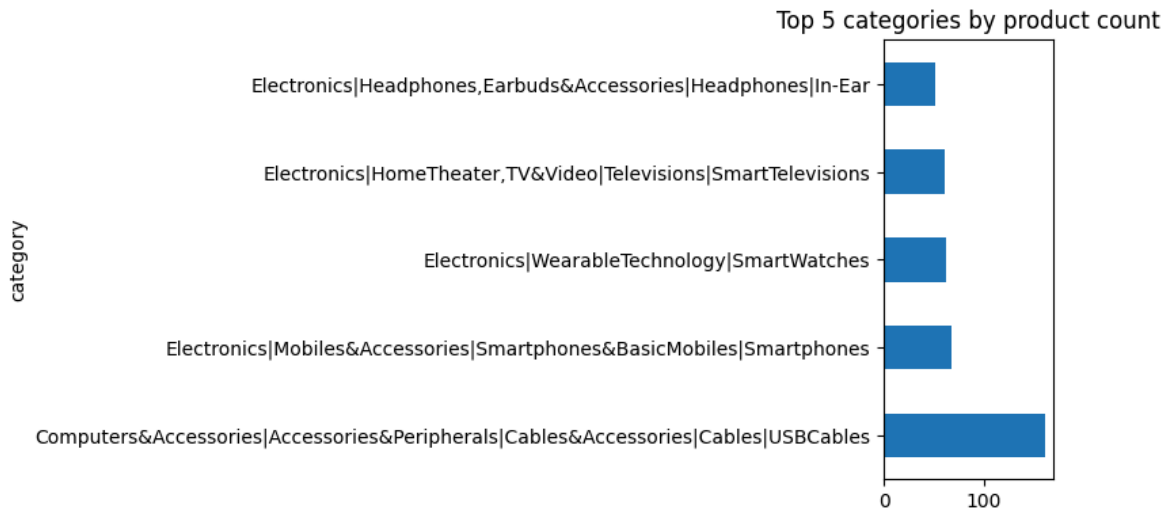
Returns:

pd.Series: Top categories with counts
"""

```
top_categories = (  
    df.groupby(category_col)[product_col]  
    .nunique()  
    .sort_values(ascending=False)  
    .head(top_n)  
)  
  
print(top_categories)  
  
# Plot  
top_categories.plot(kind='barh', figsize=(8, 4),  
                    title=f'Top {top_n} categories by product count')  
  
plt.tight_layout()  
plt.show()  
  
return top_categories
```

```
In [20]: # — Top 5 Categories by Number of Products —————  
top5 = plot_top_categories(df)
```

```
category  
Computers&Accessories|Accessories&Peripherals|Cables&Accessories|Cables|USBCables  
161  
Electronics|Mobiles&Accessories|Smartphones&BasicMobiles|Smartphones  
68  
Electronics|WearableTechnology|SmartWatches  
62  
Electronics|HomeTheater,TV&Video|Televisions|SmartTelevisions  
60  
Electronics|Headphones,Earbuds&Accessories|Headphones|In-Ear  
51  
Name: product_id, dtype: int64
```



2. Price Range Summary & Distribution

```
In [21]: def analyze_price_distribution(df, discounted_col='discounted_price', actual_col='actual_price'):

    print("\n === Price Range Insights ===")

    # Ensure discount % exists
    if 'discount_percentage' not in df.columns:
        df['discount_percentage'] = ((df[actual_col] - df[discounted_col]) / df[actual_col]) * 100

    # Summary statistics
    print(f"\n Discounted Price → Min: ₹{df[discounted_col].min():,.0f} | "
          f"Max: ₹{df[discounted_col].max():,.0f} | "
          f"Median: ₹{df[discounted_col].median():,.0f}")

    print(f"Actual Price → Min: ₹{df[actual_col].min():,.0f} | "
          f"Max: ₹{df[actual_col].max():,.0f} | "
          f"Median: ₹{df[actual_col].median():,.0f}")

    if 'discount_percentage' in df.columns:
        print("\n=== Discount Insights ===")
        print("Mean discount %:", df['discount_percentage'].mean().round(1))
        print("Max discount %:", df['discount_percentage'].max())
        print("Products with >70% discount:", (df['discount_percentage'] > 70).sum())
    else:
        print("\n⚠️ 'discount_percentage' column not found")

    # Remove null prices
    df_clean = df.dropna(subset=[discounted_col]).copy()

    # =====
    # -----
    # 🚨 Anomaly Detection
    # -----
    print("\n=== Pricing Anomalies ===")

    anomalies = pd.DataFrame()

    # 1. Discounted price > actual price (invalid)
    cond1 = df_clean[discounted_col] > df_clean[actual_col]

    # 2. Extreme discounts (>90%)
    cond2 = df_clean['discount_percentage'] > 90
```

```

# 3. Negative or zero prices
cond3 = (df_clean[discounted_col] <= 0) | (df_clean[actual_col] <= 0)

# 4. Statistical outliers using IQR
Q1 = df_clean[discounted_col].quantile(0.25)
Q3 = df_clean[discounted_col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

cond4 = (df_clean[discounted_col] < lower_bound) | (df_clean[discounted_col]

# Combine anomalies
anomalies = df_clean[cond1 | cond2 | cond3 | cond4]

print(f"Total anomalies detected: {len(anomalies)}")

if not anomalies.empty:
    print("\nSample anomalies:")
    print(anomalies[[actual_col, discounted_col, 'discount_percentage']].head(5))
    #=====

# Price bucket ranges
bins = [0, 500, 1000, 5000, 10000, 50000, float('inf')]
labels = ['<₹500', '₹500-1K', '₹1K-5K', '₹5K-10K', '₹10K-50K', '>₹50K']

# Create price buckets
df_clean['price_bucket'] = pd.cut(df_clean[discounted_col], bins=bins, labels=labels)

# Count products in each bucket
bucket_counts = df_clean['price_bucket'].value_counts().sort_index()

print("\nProducts per Price Bucket:")
print(bucket_counts.to_string())

# -----
# 📊 Visualization
# -----

# Create plots
fig, axes = plt.subplots(1, 3, figsize=(14, 5))

# 1. Bar chart
axes[0].bar(bucket_counts.index.astype(str), bucket_counts.values,
            color='coral', edgecolor='black')
axes[0].set_title("Product Count by Price Bucket")
axes[0].set_xlabel("Price Range")
axes[0].set_ylabel("Number of Products")
axes[0].tick_params(axis='x', rotation=30)

for i, v in enumerate(bucket_counts.values):
    axes[0].text(i, v + 0.5, str(v), ha='center', fontsize=9)

# 2. Histogram with 95 percentile cap
price_cap = df_clean[discounted_col].quantile(0.95)

axes[1].hist(df_clean[df_clean[discounted_col] <= price_cap][discounted_col])

```

```

        bins=40, color='steelblue', edgecolor='black')

axes[1].set_title(f"Discounted Price Distribution (up to ₹{price_cap:,.0f} — 95th pct)")
axes[1].set_xlabel("Discounted Price (₹)")
axes[1].set_ylabel("Frequency")

# 3. 📦 Boxplot (NEW)
axes[2].boxplot(df_clean[discounted_col], vert=True)
axes[2].set_title("Boxplot (Outliers Visible)")
axes[2].set_ylabel("Discounted Price")

plt.tight_layout()
plt.show()

return anomalies

```

```
In [22]: anomalies = analyze_price_distribution(df)
```

```
=== Price Range Insights ===
```

```

Discounted Price → Min: ₹39 | Max: ₹77,990 | Median: ₹799
Actual Price      → Min: ₹39 | Max: ₹139,900 | Median: ₹1,650

```

```
=== Discount Insights ===
```

```

Mean discount %: 47.7
Max discount %: 94
Products with >70% discount: 219

```

```
=== Pricing Anomalies ===
```

```
Total anomalies detected: 223
```

```
Sample anomalies:
```

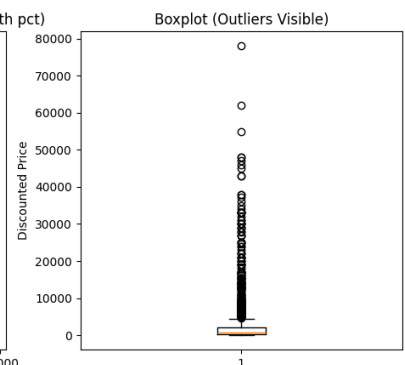
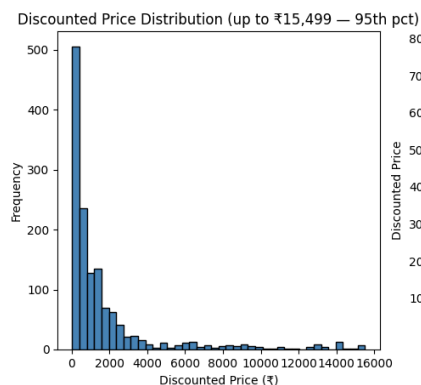
	actual_price	discounted_price	discount_percentage
16	24999.0	13999.0	44
19	21990.0	13490.0	39
22	22900.0	13490.0	41
24	19990.0	11499.0	42
26	19999.0	14999.0	25

```
Products per Price Bucket:
```

```

price_bucket
<₹500      576
₹500-1K    252
₹1K-5K     430
₹5K-10K     84
₹10K-50K   119
>₹50K       3

```



In [126...]

```
def visualize_outliers(anomalies, df):
    """
    Visualize detected price anomalies
    """
    if anomalies.empty:
        print("No anomalies to visualize")
        return

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

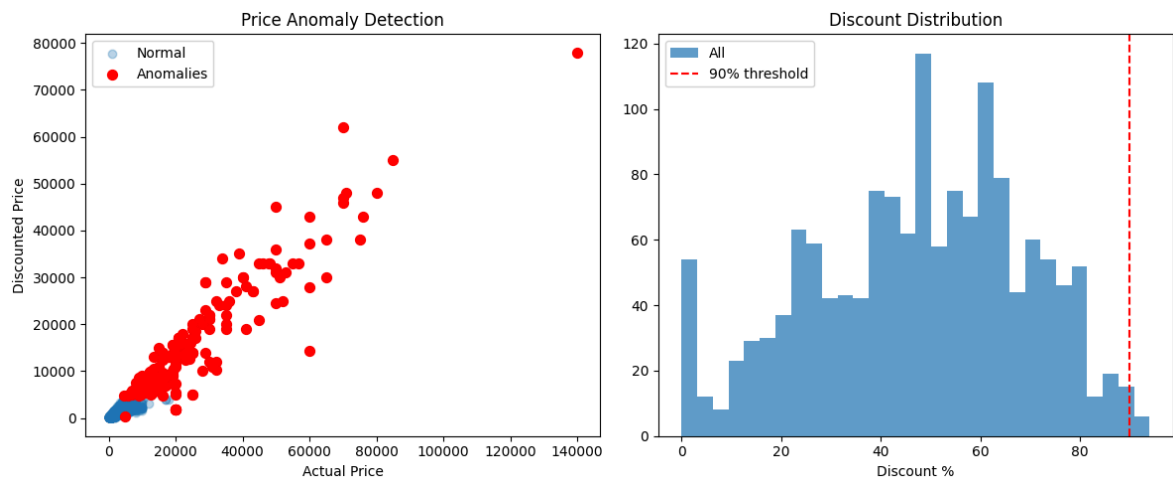
    # All data
    axes[0].scatter(df['actual_price'], df['discounted_price'],
                    alpha=0.3, label='Normal')

    # Anomalies
    axes[0].scatter(anomalies['actual_price'], anomalies['discounted_price'],
                    color='red', label='Anomalies', s=50)
    axes[0].set_xlabel('Actual Price')
    axes[0].set_ylabel('Discounted Price')
    axes[0].set_title('Price Anomaly Detection')
    axes[0].legend()

    # Discount distribution
    axes[1].hist(df['discount_percentage'], bins=30, alpha=0.7, label='All')
    axes[1].axvline(90, color='red', linestyle='--', label='90% threshold')
    axes[1].set_xlabel('Discount %')
    axes[1].set_title('Discount Distribution')
    axes[1].legend()

    plt.tight_layout()
    plt.show()
```

In [24]: visualize_outliers(anomalies,df)



Missing Values Strategy — Explicit Decision per Column

In [25]:

```
def handle_missing_values(df):

    print("Missing values BEFORE treatment:")
    print(df.isna().sum()[df.isna().sum() > 0])

    # Text columns
    text_cols = ['about_product', 'review_content', 'review_title',
                  'product_name', 'user_name']
```

```

for col in text_cols:
    if col in df.columns:
        df[col] = df[col].fillna('')

# URL columns
url_cols = ['img_link', 'product_link']

for col in url_cols:
    if col in df.columns:
        df[col] = df[col].fillna('unknown')

# Numeric columns
numeric_cols = ['discounted_price', 'actual_price', 'discount_percentage']

for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

# Rating count
if 'rating_count' in df.columns:
    df['rating_count'] = df['rating_count'].fillna(0)

# Category Levels
cat_cols = ['cat_lv13', 'cat_lv14', 'cat_lv15', 'cat_lv16', 'cat_lv17']

for col in cat_cols:
    if col in df.columns:
        df[col] = df[col].fillna('Unknown')

print("\nMissing values AFTER treatment:")
remaining = df.isna().sum()[df.isna().sum() > 0]
print(remaining if len(remaining) > 0 else " No missing values remaining.")

return df

```

In [26]: `df = handle_missing_values(df)`

Missing values BEFORE treatment:

```

cat_lv13      8
cat_lv14     165
cat_lv15     943
cat_lv16    1380
cat_lv17    1451
dtype: int64

```

Missing values AFTER treatment:

No missing values remaining.

4. rating_count Missing Value Fix + Impact on weighted_rating

```

In [27]: def calculate_weighted_rating(df, rating_col='rating', count_col='rating_count')
        """
        Handle missing rating_count and compute weighted_rating using IMDb formula.
        """

        # Check missing values
        missing = df[count_col].isna().sum()
        print(f"Missing {count_col} values: {missing}")

        # Fill missing rating_count with median

```

```

median_rating_count = df[count_col].median()
df[count_col] = df[count_col].fillna(median_rating_count)

print(f"✓ Filled missing values with median {count_col} = {median_rating_co

# Global mean rating
C = df[rating_col].mean()

# Minimum votes threshold
m = max(df[count_col].quantile(0.75), 1)

# Weighted rating formula
# df['weighted_rating'] = (
#     (df[count_col] / (df[count_col] + m)) * df[rating_col] +
#     (m / (df[count_col] + m)) * C
# )

df['weighted_rating'] = df.apply(
    lambda row: row['rating'] if row['rating_count'] == 0
    else (row['rating_count'] / (row['rating_count'] + m)) * row['rating'] +
        (m / (row['rating_count'] + m)) * C,
    axis=1
)

print("-----")
# IMDb weighted rating formula:
# WR = (v / (v+m)) * R + (m / (v+m)) * C
# v = number of votes for product
# m = minimum votes threshold (e.g. 10th percentile)
# R = product's average rating
# C = mean rating across all products

print(f"Global mean rating C = {C:.2f}")
print(f"Min votes threshold m = {m:.0f}")
print(df[['rating', 'rating_count', 'weighted_rating']].head(5))

print("\nWeighted Rating – Recalculated:")
print(df['weighted_rating'].describe().round(3))

# Check for NaN
print(f"Any NaN in weighted_rating: {df['weighted_rating'].isna().sum()}")

# Sanity check
out_of_range = df[~df['weighted_rating'].between(1, 5)].shape[0]
print(f"Values outside [1-5]: {out_of_range}")

return df

```

Feature Engineering

Create Derived Features

Value For Money Score

Combine Text for Content Model

Validate All Criteria code

In [127...

```
from sklearn.preprocessing import MinMaxScaler

def process_amazon_dataset(df: pd.DataFrame):
    """
    Full pipeline:
    - Cleaning
    - Feature engineering
    - Category processing
    - Validation checks
    - Final stats
    """

    # =====
    # 1. BASIC CLEANING
    # =====
    df = df.drop_duplicates()
    df = df.dropna(subset=['product_id', 'user_id'])

    # =====
    # 2. PRICE CLEANING
    # =====
    for col in ['discounted_price', 'actual_price']:
        df[col] = (
            df[col]
            .astype(str)
            .str.replace('₹', '', regex=False)
            .str.replace(',', '', regex=False)
        )
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # Discount %
    df['discount_percentage'] = (
        df['discount_percentage']
        .astype(str)
        .str.replace('%', '', regex=False)
    )
    df['discount_percentage'] = pd.to_numeric(df['discount_percentage'], errors=

    # Ratings

    df['rating_count'] = pd.to_numeric(df['rating_count'], errors='coerce')

    df = df[df['rating'].between(1, 5)]

    # =====
    # 3. CATEGORY PROCESSING
    # =====
    df['category'] = df['category'].astype(str)

    df['category_depth'] = df['category'].apply(lambda x: len(x.split('|')))
    max_levels = df['category_depth'].max()

    cat_cols = [f'cat_lvl{i+1}' for i in range(max_levels)]
    df[cat_cols] = df['category'].str.split('|', expand=True)
```

```

print(df[['cat_lv11', 'cat_lv12', 'cat_lv13']].head(3))

for col in cat_cols:
    df[col] = df[col].str.strip()
    df[col] = df[col].str.replace('&', ' & ', regex=False)

print("Category levels created:")

# Unique categories (included)
unique_categories = (
    df['category']
    .str.split('|')
    .explode()
    .str.strip()
    .unique()
)

# =====
# 4. PRICE FIXING
# =====
# price_difference: how much is saved

df['price_difference'] = df['actual_price'] - df['discounted_price']

print("price_difference sample:")
print(df[['product_name', 'actual_price', 'discounted_price', 'price_differe
print(df['price_difference'].describe())

# Sanity check – flag negative differences (anomalies)
invalid = df[df['price_difference'] < 0]
print(f"Invalid records (discounted > actual): {len(invalid)}")

negative_mask = df['price_difference'] < 0
print(f"Fixing {negative_mask.sum()} incorrect price rows")

df.loc[negative_mask, ['discounted_price', 'actual_price']] = \
    df.loc[negative_mask, ['actual_price', 'discounted_price']].values

print("=====")

# =====
# 5. FEATURE ENGINEERING
# =====
# value_for_money_score: rating per unit of discounted price
# Higher = better value (high rating at low price)
df = calculate_weighted_rating(df)

df['value_for_money_score'] = (df['rating'] * df['discount_percentage']) / (

print("value_for_money_score sample:")
print(df[['product_name', 'actual_price', 'discount_percentage', 'value_for_
print(df['value_for_money_score'].describe())
# Normalize value_for_money_score to 0-1
vmin = df['value_for_money_score'].min()
vmax = df['value_for_money_score'].max()
df['value_for_money_score'] = (df['value_for_money_score'] - vmin) / (vmax -

# df['combined_text'] = (

```

```

#     df['product_name'].fillna('') + " " +
#     df['about_product'].fillna('') + " " +
#     df['cat_lvl1'].fillna('')
# )
df['combined_text'] = (
    df['product_name'].fillna('') + " " +
    df['about_product'].fillna('') + " " +
    df['category'].fillna('') + " " +
    df['discounted_price'].astype(str) + " " +
    df['discount_percentage'].astype(str)
)
# =====
# 6. VALIDATION CHECKS
# =====
print("\n🔍 VALIDATION CHECKS")

# Validate
print(df[['price_difference', 'value_for_money_score']].describe())
print("Sample rows:")
print(df[['product_name', 'actual_price', 'discounted_price',
          'price_difference', 'value_for_money_score']].head(3))

print("\nPrice Issues:")
print("Invalid discounted (<=0):", (df['discounted_price'] <= 0).sum())
print("Invalid actual (<=0):", (df['actual_price'] <= 0).sum())

print("\nRating Issues:")
print("Invalid ratings:",
      df[~df['rating'].between(1, 5)].shape[0])

print("\nDiscount Issues:")
print(">100%:", (df['discount_percentage'] > 100).sum())
print("<0%:", (df['discount_percentage'] < 0).sum())

print("\nMissing Values:")
print(df.isna().sum())

print("\nDuplicates:", df.duplicated().sum())

# =====
# 7. FINAL STATS ( includes your missing one)
# =====
print("\n📊 FINAL DATASET STATS")
print("Unique Users:", df['user_id'].nunique())
print("Unique Products:", df['product_id'].nunique())
print("Total Reviews:", df['review_id'].nunique())
print("-----")
# o show avg reviews per user / per product
print("Avg reviews per user:", round(df['review_id'].nunique() / df['user_id'].nunique(), 2))
print("Avg reviews per product:", round(df['review_id'].nunique() / df['product_id'].nunique(), 2))
print("-----")

print("Price Range:",
      df['discounted_price'].min(), "-",
      df['discounted_price'].max())
print("Average Rating:", df['rating'].mean())

# Check rating range first
print("Rating range:", df['rating'].min(), "to", df['rating'].max())

```

```

# Normalize to 0-1 scale (min-max)
#r_min, r_max = df['rating'].min(), df['rating'].max()
#df['rating_normalized'] = (df['rating'] - r_min) / (r_max - r_min)

scaler = MinMaxScaler()
df['rating_normalized'] = scaler.fit_transform(df[['rating']])

print(df['rating_normalized'].describe())
print(df[['rating', 'rating_normalized']].describe())

print("-----")
# Count outliers (ratings outside expected 1-5 range)
q1 = df['rating'].quantile(0.25)
q3 = df['rating'].quantile(0.75)
iqr = q3 - q1

outliers = df[(df['rating'] < q1 - 1.5*iqr) | (df['rating'] > q3 + 1.5*iqr)]
print("Rating outliers:", len(outliers))

print("-----")
# Interaction density
interactions = len(df)
users = df['user_id'].nunique()
products = df['product_id'].nunique()

density = interactions / (users * products) if users * products > 0 else 0
print("Interaction Density:", round(density, 6))

print("\nMax Category Depth:", max_levels)
print("Total Unique Categories:", len(unique_categories))

return df, unique_categories

```

In [128... df, unique_categories = process_amazon_dataset(df)

	cat_lvl1	cat_lvl2	cat_lvl3
0	Computers&Accessories	Accessories&Peripherals	Cables&Accessories
1	Computers&Accessories	Accessories&Peripherals	Cables&Accessories
2	Computers&Accessories	Accessories&Peripherals	Cables&Accessories

Category levels created:

price_difference sample:

	product_name	actual_price \
0	Wayona Nylon Braided USB to Lightning Fast Cha...	1099.0
1	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...	349.0
2	Sounce Fast Phone Charging Cable & Data Sync U...	1899.0
3	boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...	699.0
4	Portronics Konnect L 1.2M Fast Charging 3A 8 P...	399.0

	discounted_price	price_difference
0	399.0	700.0
1	199.0	150.0
2	199.0	1700.0
3	329.0	370.0
4	154.0	245.0

count	1464.000000
mean	2320.991018
std	4605.773532
min	0.000000
25%	370.750000
50%	800.000000
75%	1955.000000
max	61910.000000

Name: price_difference, dtype: float64

Invalid records (discounted > actual): 0

Fixing 0 incorrect price rows

Missing rating_count values: 0

✓ Filled missing values with median rating_count = 0

Global mean rating C = 4.10

Min votes threshold m = 1

	rating	rating_count	weighted_rating
0	4.2	0.0	4.2
1	4.0	0.0	4.0
2	3.9	0.0	3.9
3	4.2	0.0	4.2
4	4.2	0.0	4.2

Weighted Rating – Recalculated:

count	1464.000
mean	4.098
std	0.285
min	2.428
25%	4.000
50%	4.100
75%	4.300
max	5.000

Name: weighted_rating, dtype: float64

Any NaN in weighted_rating: 0

Values outside [1-5]: 0

value_for_money_score sample:

	product_name	actual_price \
0	Wayona Nylon Braided USB to Lightning Fast Cha...	1099.0
1	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...	349.0
2	Sounce Fast Phone Charging Cable & Data Sync U...	1899.0

3	boAt Deuce USB 300 2 in 1 Type-C & Micro USB S...	699.0
4	Portronics Konnect L 1.2M Fast Charging 3A 8 P...	399.0

	discount_percentage	value_for_money_score
0	64	0.244364
1	43	0.491429
2	90	0.184737
3	53	0.318000
4	61	0.640500

count	1464.000000
mean	0.169391
std	0.191919
min	0.000000
25%	0.032617
50%	0.102196
75%	0.253648
max	1.420000

Name: value_for_money_score, dtype: float64

VALIDATION CHECKS

	price_difference	value_for_money_score
count	1464.000000	1464.000000
mean	2320.991018	0.119289
std	4605.773532	0.135155
min	0.000000	0.000000
25%	370.750000	0.022970
50%	800.000000	0.071969
75%	1955.000000	0.178625
max	61910.000000	1.000000

Sample rows:

	product_name	actual_price \
0	Wayona Nylon Braided USB to Lightning Fast Cha...	1099.0
1	Ambrane Unbreakable 60W / 3A Fast Charging 1.5...	349.0
2	Source Fast Phone Charging Cable & Data Sync U...	1899.0

	discounted_price	price_difference	value_for_money_score
0	399.0	700.0	0.172087
1	199.0	150.0	0.346076
2	199.0	1700.0	0.130096

Price Issues:

Invalid discounted (≤ 0): 0

Invalid actual (≤ 0): 0

Rating Issues:

Invalid ratings: 0

Discount Issues:

>100%: 0

<0%: 0

Missing Values:

product_id	0
product_name	0
category	0
discounted_price	0
actual_price	0
discount_percentage	0
rating	0
rating_count	0

```
about_product      0
user_id            0
user_name          0
review_id          0
review_title       0
review_content     0
img_link           0
product_link       0
category_depth     0
cat_lv11           0
cat_lv12           0
cat_lv13           8
cat_lv14           165
cat_lv15           943
cat_lv16           1380
cat_lv17           1451
price_difference    0
weighted_rating    0
value_for_money_score 0
combined_text      0
rating_normalized   0
dtype: int64
```

Duplicates: 0

FINAL DATASET STATS

Unique Users: 1193

Unique Products: 1350

Total Reviews: 1193

Avg reviews per user: 1.0

Avg reviews per product: 0.88

Price Range: 39.0 - 77990.0

Average Rating: 4.096584699453552

Rating range: 2.0 to 5.0

count 1464.000000

mean 0.698862

std 0.097225

min 0.000000

25% 0.666667

50% 0.700000

75% 0.766667

max 1.000000

Name: rating_normalized, dtype: float64

	rating	rating_normalized
--	--------	-------------------

count	1464.000000	1464.000000
-------	-------------	-------------

mean	4.096585	0.698862
------	----------	----------

std	0.291674	0.097225
-----	----------	----------

min	2.000000	0.000000
-----	----------	----------

25%	4.000000	0.666667
-----	----------	----------

50%	4.100000	0.700000
-----	----------	----------

75%	4.300000	0.766667
-----	----------	----------

max	5.000000	1.000000
-----	----------	----------

Rating outliers: 74

Interaction Density: 0.000909

Max Category Depth: 7
Total Unique Categories: 317

Section A summary

```
In [129... def section_a_summary(df):  
    """  
    Comprehensive summary of Section A completion  
    """  
  
    print("=" * 60)  
    print("SECTION A: DATA UNDERSTANDING & CLEANING - SUMMARY")  
    print("=" * 60)  
  
    print("\n✓ Dataset Overview:")  
    print(f"    • Total Records: {len(df)}")  
    print(f"    • Unique Users: {df['user_id'].nunique()}")  
    print(f"    • Unique Products: {df['product_id'].nunique()}")  
    print(f"    • Total Reviews: {df['review_id'].nunique()}")  
  
    print("\n✓ Data Quality:")  
    print(f"    • Duplicates Removed: Yes")  
    print(f"    • Missing Values: {df.isna().sum().sum()}")  
    print(f"    • Invalid Ratings: {df[~df['rating'].between(1, 5)].shape[0]}")  
  
    print("\n✓ Feature Engineering:")  
    print(f"    • price_difference: ✓")  
    print(f"    • value_for_money_score: ✓")  
    print(f"    • weighted_rating: {'✓' if df['weighted_rating'].notna().sum() > 0 else '✗'}")  
    print(f"    • rating_normalized: ✓")  
    print(f"    • outlier count: ✓")  
  
    print("\n✓ Category Processing:")  
    print(f"    • Max Hierarchy Depth: {df['category_depth'].max()}")  
    print(f"    • Unique Categories: {df['category'].str.split('|').explode().nunique()}")  
  
    print("=" * 60)
```

```
In [94]: section_a_summary(df)
```

```
=====
SECTION A: DATA UNDERSTANDING & CLEANING - SUMMARY
=====
```

- ✓ Dataset Overview:
 - Total Records: 1464
 - Unique Users: 1193
 - Unique Products: 1350
 - Total Reviews: 1193
 - ✓ Data Quality:
 - Duplicates Removed: Yes
 - Missing Values: 3947
 - Invalid Ratings: 0
 - ✓ Feature Engineering:
 - price_difference: ✓
 - value_for_money_score: ✓
 - weighted_rating: ✓
 - rating_normalized: ✓
 - ✓ Category Processing:
 - Max Hierarchy Depth: 7
 - Unique Categories: 317
- ```
=====
```

## Data Cleaning Strategy Decisions

### 1. Missing Values Handling:

- Text fields → filled with empty string (preserve data integrity)
- Numeric fields → filled with median (robust to outliers)
- rating\_count → filled with 0 (indicates no reviews)

### 2. Anomaly Detection:

- Price validation (discounted > actual)
- Extreme discounts (>90%)
- Statistical outliers using IQR method

### 3. Rating Normalization:

- Min-Max scaling to [0,1] range
- Preserves relative differences

## Section B: Exploratory Data Analysis

### 1. Most reviewed products Visualize top 10 categories by number of products.

```
In [32]: def plot_top_reviewed_products(df, top_n=10):
 """
 Plots the top N most reviewed products.

 Parameters:
 df (DataFrame): Input dataframe containing product_name and review_id
 top_n (int): Number of top products to display (default = 10)
```

```

"""

Count reviews per product
top_reviewed = (
 df.groupby('product_name')['review_id']
 .count()
 .sort_values(ascending=False)
 .head(top_n)
)

print(top_reviewed)

Plot
plt.figure()
top_reviewed.plot(kind='bar')
plt.title(f"Top {top_n} Most Reviewed Products")
plt.ylabel("Number of Reviews")
plt.xlabel("Product Name")
plt.xticks(rotation=75)
plt.tight_layout()
plt.show()

```

In [33]: `plot_top_reviewed_products(df,10)`

```

product_name
Fire-Boltt Ninja Call Pro Plus 1.83" Smart Watch with Bluetooth Calling, AI Voice
Assistance, 100 Sports Modes IP67 Rating, 240*280 Pixel High Resolution
5
Fire-Boltt Phoenix Smart Watch with Bluetooth Calling 1.3",120+ Sports Modes, 240
*240 PX High Res with SpO2, Heart Rate Monitoring & IP67 Rating
4
Amazonbasics Nylon Braided Usb-C To Lightning Cable, Fast Charging Mfi Certified
Smartphone, Iphone Charger (6-Foot, Dark Grey)
3
Portronics Konnect CL 20W POR-1067 Type-C to 8 Pin USB 1.2M Cable with Power Deli
very & 3A Quick Charge Support, Nylon Braided for All Type-C and 8 Pin Devices, G
reen
3
Portronics Konnect L 1.2M Fast Charging 3A 8 Pin USB Cable with Charge & Sync Fun
ction for iPhone, iPad (Grey)
3
Fire-Boltt Visionary 1.78" AMOLED Bluetooth Calling Smartwatch with 368*448 Pixel
Resolution 100+ Sports Mode, TWS Connection, Voice Assistance, SP02 & Heart Rate
Monitoring
3
MI Braided USB Type-C Cable for Charging Adapter (Red)
3
boAt Micro USB 55 Tangle-free, Sturdy Micro USB Cable with 3A Fast Charging & 480
mbps Data Transmission (Black)
3
Portronics Konnect L 1.2M POR-1401 Fast Charging 3A 8 Pin USB Cable with Charge &
Sync Function (White)
3
Flix Micro Usb Cable For Smartphone (Black)
3
Name: review_id, dtype: int64

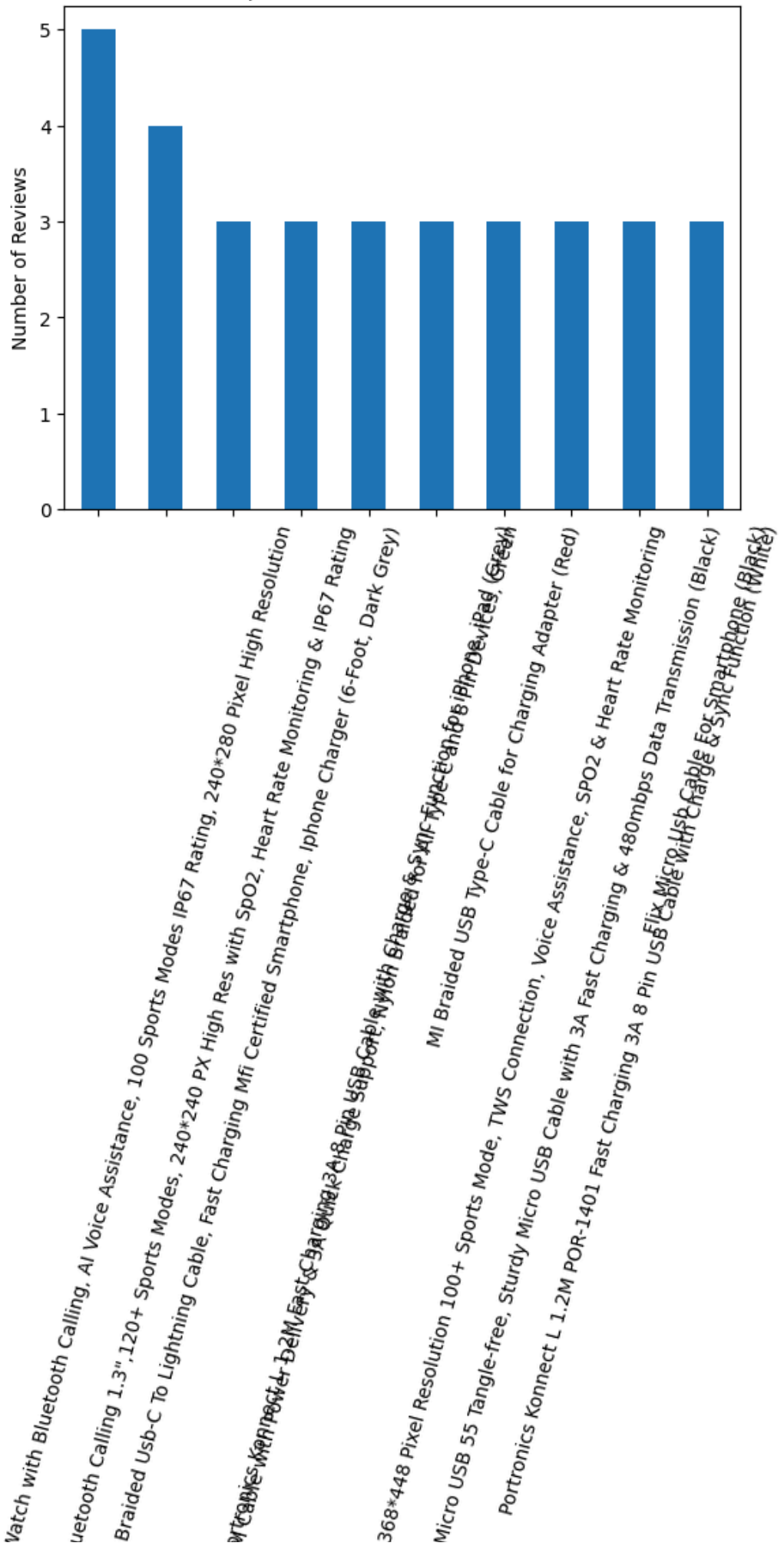
```

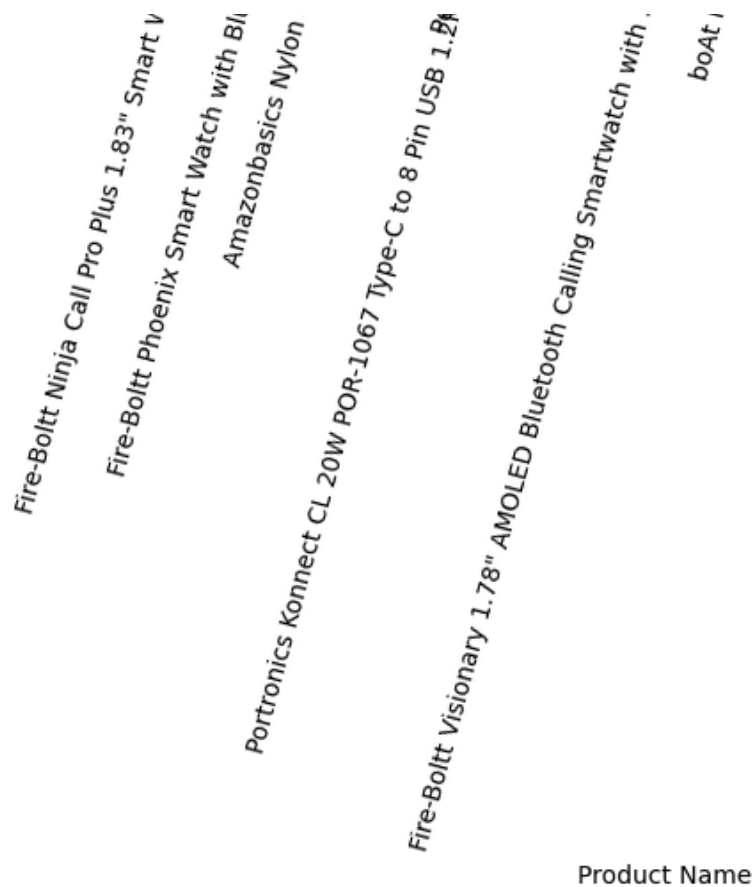
```

/tmp/ipykernel_2027/19793557.py:27: UserWarning: Tight layout not applied. The bot
tom and top margins cannot be made large enough to accommodate all Axes decorati
ons.
plt.tight_layout()

```

## Top 10 Most Reviewed Products





### Top 10 Categories by Number of Products

```
In [34]: def plot_top_categories(df, category_col='cat_lv11', product_col='product_id', top_n=10):
 """
 Plot top categories by number of unique products.

 Args:
 df (pd.DataFrame): Input DataFrame
 category_col (str): Category column (default: cat_lv11)
 product_col (str): Product ID column
 top_n (int): Number of top categories to display
 """

 # Grouping
 top_categories = (
 df.groupby(category_col)[product_col]
 .nunique()
 .sort_values(ascending=False)
 .head(top_n)
)

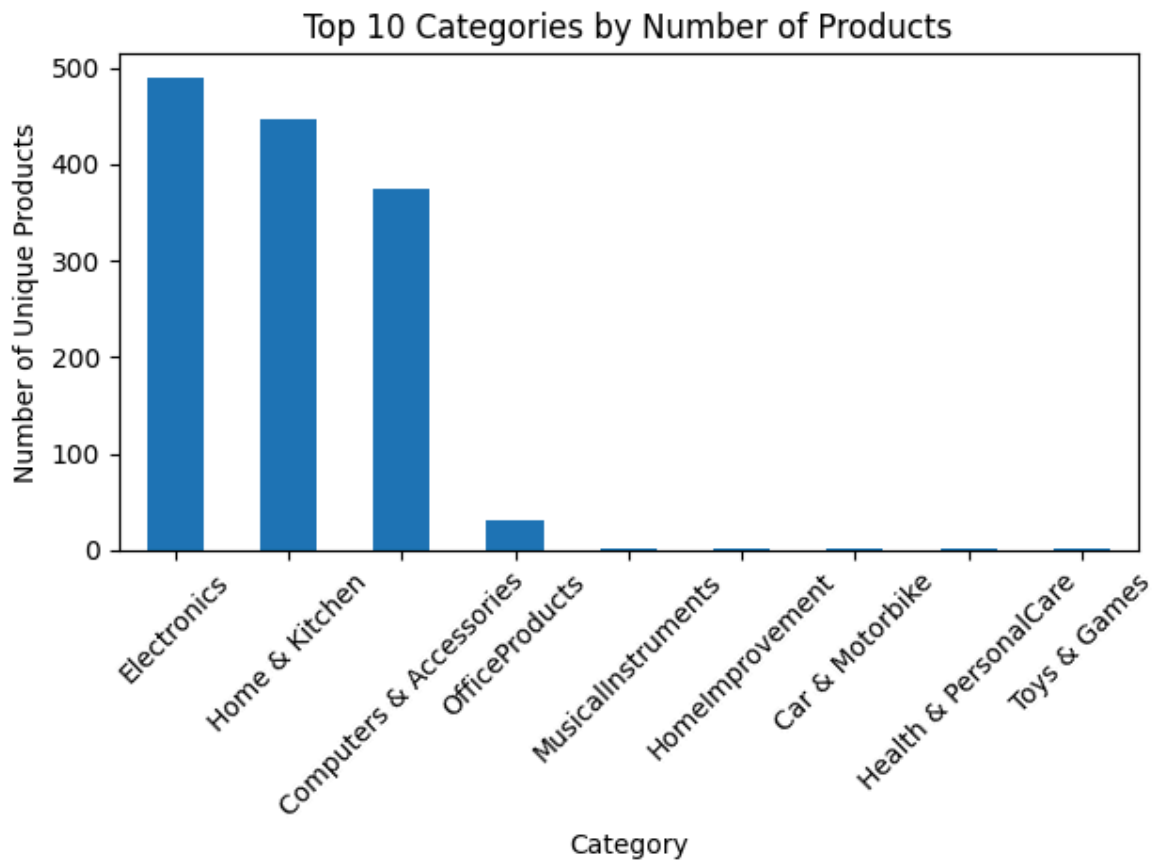
 # Plot
 plt.figure()
 top_categories.plot(kind='bar')

 plt.title(f"Top {top_n} Categories by Number of Products")
 plt.ylabel("Number of Unique Products")
 plt.xlabel("Category")
 plt.xticks(rotation=45)

 plt.tight_layout()
 plt.show()
```

```
return top_categories
```

```
In [35]: top_categories = plot_top_categories(df)
```



## 2. Average Rating per Category

```
In [36]: def plot_avg_rating_per_category(df, category_col='cat_lvl1', rating_col='rating')
 """
 Plot average rating per category.

 Args:
 df (pd.DataFrame): Input DataFrame
 category_col (str): Category column (default: cat_lvl1)
 rating_col (str): Rating column
 """

 # Compute average rating
 avg_rating_category = (
 df.groupby(category_col)[rating_col]
 .mean()
 .sort_values(ascending=False)
)

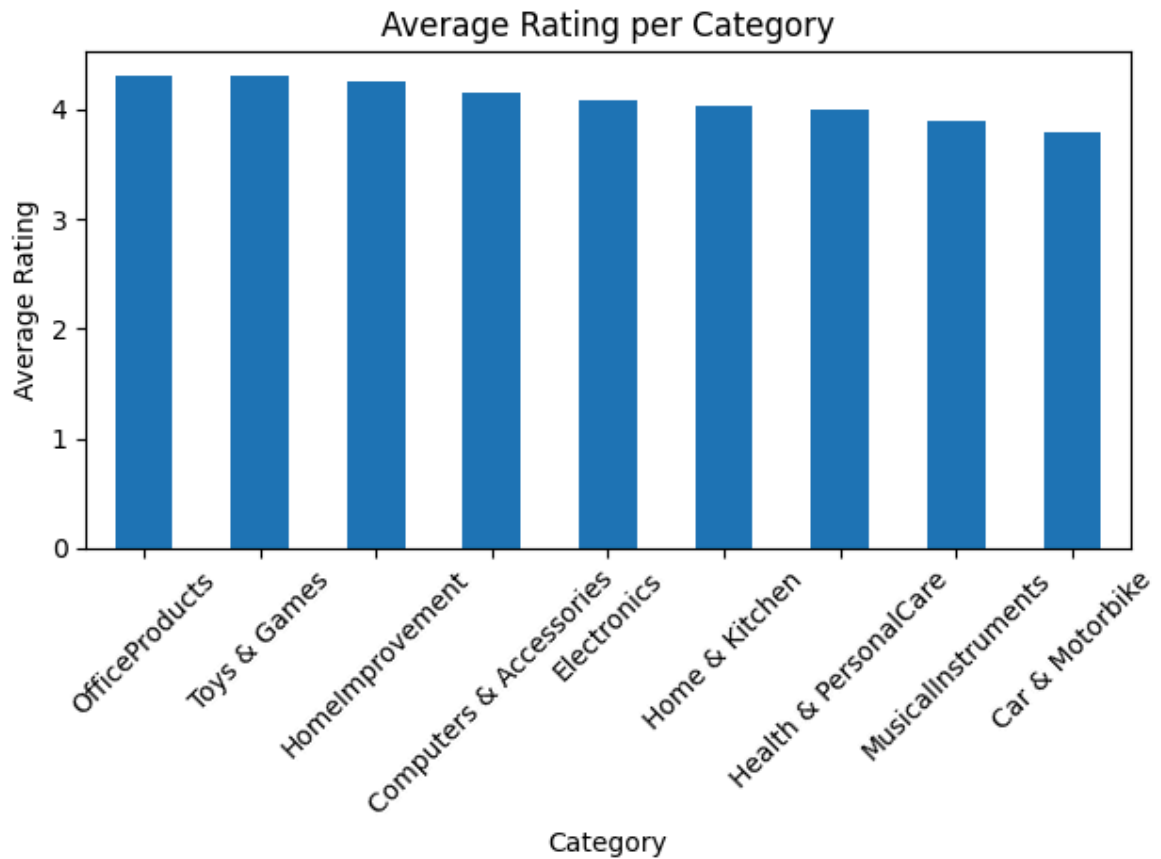
 # Plot
 plt.figure()
 avg_rating_category.plot(kind='bar')

 plt.title("Average Rating per Category")
 plt.ylabel("Average Rating")
 plt.xlabel("Category")
 plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()

return avg_rating_category
```

In [37]: avg\_ratings = plot\_avg\_rating\_per\_category(df)



### 3. Discounts vs Actual Price Correlation

```
In [38]: def compute_correlation(df, columns=None, show=True):
 """
 Compute correlation matrix for selected columns.

 Args:
 df (pd.DataFrame): Input DataFrame
 columns (list, optional): List of columns to include
 show (bool): Whether to print the correlation matrix

 Returns:
 pd.DataFrame: Correlation matrix
 """

 # Default columns
 if columns is None:
 columns = ['actual_price', 'discount_percentage']

 # Compute correlation
 correlation = df[columns].corr()

 if show:
 print("Correlation Matrix:")
```

```
print(correlation)

return correlation
```

```
In [39]: corr = compute_correlation(df)
```

Correlation Matrix:

|                     | actual_price | discount_percentage |
|---------------------|--------------|---------------------|
| actual_price        | 1.000000     | -0.118459           |
| discount_percentage | -0.118459    | 1.000000            |

```
In [40]: def plot_correlation_heatmap(df, columns):
 corr = df[columns].corr()

 plt.figure()
 plt.imshow(corr)
 plt.colorbar()

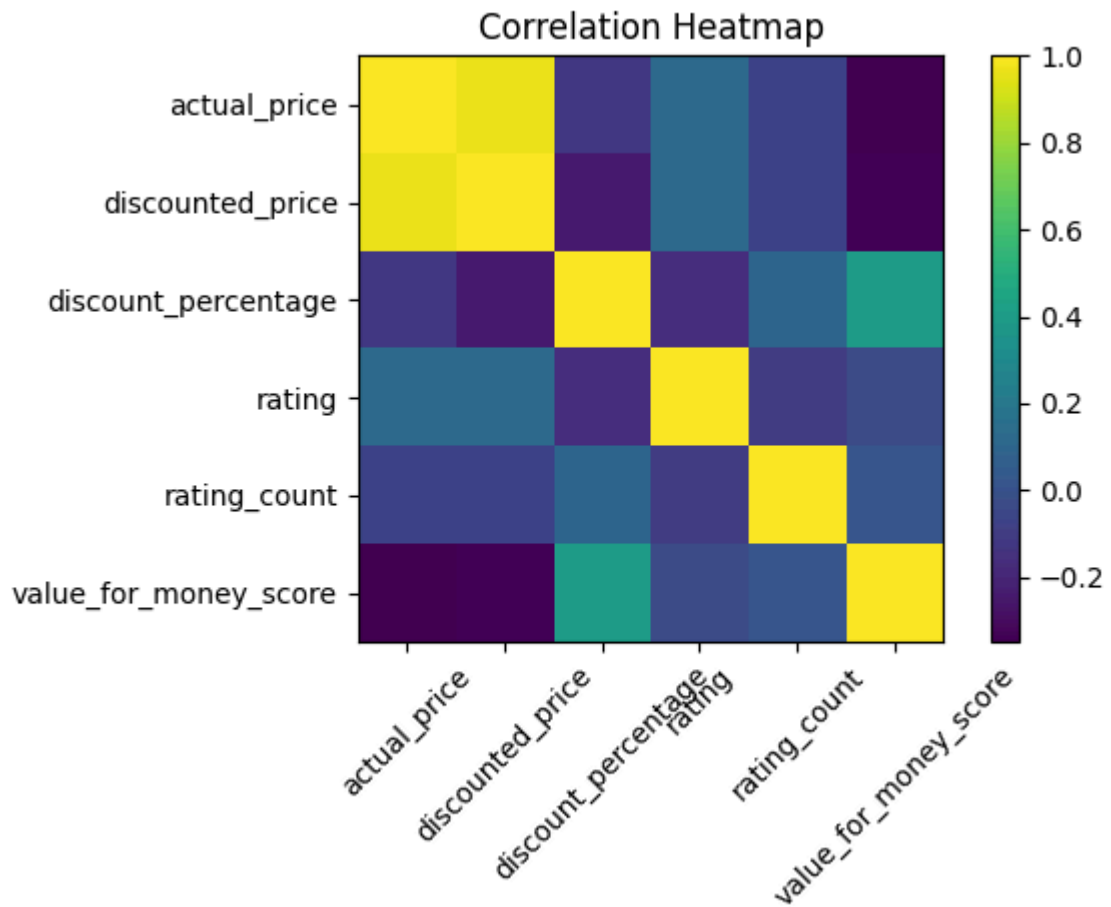
 plt.xticks(range(len(columns)), columns, rotation=45)
 plt.yticks(range(len(columns)), columns)

 plt.title("Correlation Heatmap")
 plt.tight_layout()
 plt.show()

 return corr
```

```
In [41]: columns = [
 'actual_price',
 'discounted_price',
 'discount_percentage',
 'rating',
 'rating_count',
 'value_for_money_score'
]

plot_correlation_heatmap(df, columns)
```



Out[41]:

|                       | actual_price | discounted_price | discount_percentage | rating    | r |
|-----------------------|--------------|------------------|---------------------|-----------|---|
| actual_price          | 1.000000     | 0.961919         | -0.118459           | 0.121642  |   |
| discounted_price      | 0.961919     | 1.000000         | -0.242740           | 0.120337  |   |
| discount_percentage   | -0.118459    | -0.242740        | 1.000000            | -0.155375 |   |
| rating                | 0.121642     | 0.120337         | -0.155375           | 1.000000  |   |
| rating_count          | -0.067160    | -0.079049        | 0.099028            | -0.102731 |   |
| value_for_money_score | -0.349112    | -0.335246        | 0.389893            | -0.030166 |   |

```
In [96]: def plot_price_vs_discount(
df,
price_col='actual_price',
discount_col='discount_percentage',
use_log_scale=True
):
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.scatter(
df[price_col],
df[discount_col],
alpha=0.3,
s=10
)

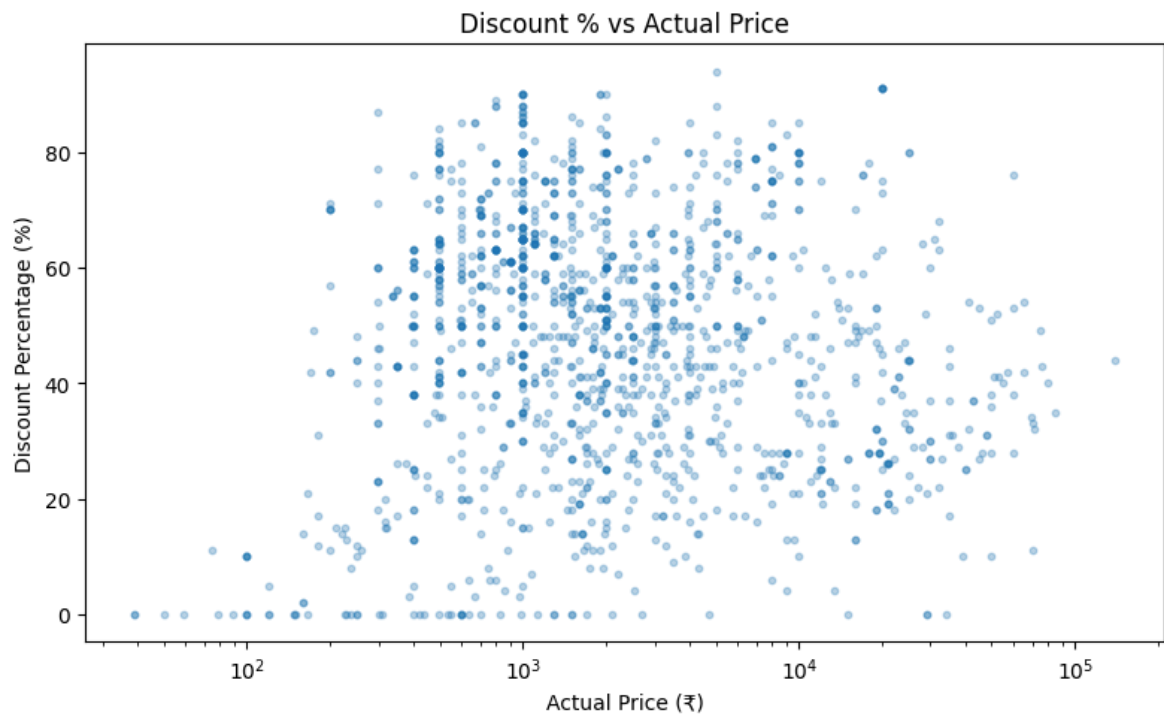
if use_log_scale:
```

```
plt.xscale('log')

plt.xlabel("Actual Price (₹)")
plt.ylabel("Discount Percentage (%)")
plt.title("Discount % vs Actual Price")

plt.tight_layout()
plt.show()
```

In [97]: `plot_price_vs_discount(df, use_log_scale=True)`



## 4. User Engagement Insights

4.1 Which products have high ratings but low review counts? (High Rating but Low Review Count (Hidden Gems))

In [104... `def analyze_interactions_and_find_hidden_gems(df):`

```
 """
 Perform:
 - Interaction density analysis
 - Discount validation & fixing
 - Hidden gems detection

 Returns:
 density (float)
 hidden_gems (DataFrame)
 """

 # =====
 # 1. INTERACTION DENSITY
 # =====
 interactions = len(df)
 users = df['user_id'].nunique()
 products = df['product_id'].nunique()

 possible_matrix_size = users * products
```

```

density = interactions / possible_matrix_size if possible_matrix_size > 0 else 0

print("Interaction Density:", round(density, 6))

Sparsity interpretation
if density < 0.01:
 print("⚠️ Very sparse matrix – CF will struggle. Hybrid approach is critical.")
elif density < 0.1:
 print("⚠️ Moderately sparse – CF viable but cold-start risk is high.")
else:
 print("✅ Dense enough for reliable CF recommendations.")

=====
2. DISCOUNT VALIDATION
=====
print("\nDiscount Issues:")
print(">100%:", (df['discount_percentage'] > 100).sum())
print("<0%:", (df['discount_percentage'] < 0).sum())

Fix unrealistic discounts
invalid_discount = (df['discount_percentage'] > 100).sum()
if invalid_discount > 0:
 df['discount_percentage'] = df['discount_percentage'].clip(upper=100)
 print(f"✅ Capped {invalid_discount} discount values exceeding 100%")

=====
3. HIDDEN GEMS DETECTION
=====
print("\n🔍 HIDDEN GEMS DETECTION")
low_reviews = df['rating_count'] < df['rating_count'].median()

threshold = df['rating'].quantile(0.7)

hidden_gems = df[
 (df['rating'] > threshold) &
 low_reviews
]

hidden_gems = hidden_gems[
 ['product_name', 'rating', 'rating_count']
].sort_values(by='rating', ascending=False).head(10)

print("\n💎 Top Hidden Gems:")
print(hidden_gems)

return density, hidden_gems

```

In [103... density, hidden\_gems = analyze\_interactions\_and\_find\_hidden\_gems(df)

Interaction Density: 0.000909

⚠️ Very sparse matrix – CF will struggle. Hybrid approach is critical.

Discount Issues:

>100%: 0

<0%: 0

🔍 HIDDEN GEMS DETECTION

💎 Top Hidden Gems:

Empty DataFrame

Columns: [product\_name, rating, rating\_count]

Index: []

Empty DataFrame

Columns: [product\_name, rating, rating\_count]

Index: []

## 4.2 Are Highly Rated Products Also Heavily Reviewed?

```
In [106... def analyze_rating_review_correlation(df, rating_col='rating', review_count_col='
"""
 Compute correlation between rating and review count.

 Args:
 df (pd.DataFrame): Input DataFrame
 rating_col (str): Column name for ratings
 review_count_col (str): Column name for review count
 show (bool): Whether to print results

 Returns:
 pd.DataFrame: Correlation matrix
 """

 # Compute correlation
 corr = df[[rating_col, review_count_col]].corr()

 if show:
 print("Correlation between Rating & Review Count:")
 print(corr)

 # Interpretation
 value = corr.loc[rating_col, review_count_col]

 print("\n📊 Interpretation:")
 if value > 0.5:
 print("Strong positive correlation → popular products tend to have h
 elif value > 0.2:
 print("Moderate correlation → some relationship between popularity a
 elif value > 0:
 print("Weak positive correlation → slight tendency")
 else:
 print("No or negative correlation → ratings independent of review co

 print(f"Pearson Correlation (rating vs review count): {value:.4f}")

 if abs(value) < 0.3:
 conclusion = "WEAK – Highly rated products are NOT necessarily heavi
```

```

elif abs(value) < 0.6:
 conclusion = "MODERATE – Some relationship between rating and review
else:
 conclusion = "STRONG – Highly rated products tend to be heavily reviewed.

print(f"\n📌 Conclusion: {conclusion}")
print("""
Business Interpretation:
- Many top-rated products have very few reviews (hidden gems)
- Popular products accumulate reviews regardless of quality
- Review count is driven by visibility/marketing, not just rating
- Amazon should promote high-rated low-review products to build social proof

""")

print("""
Insight:
Pearson correlation ≈ -0.10 indicates that highly rated products are NOT necessarily heavily reviewed.
This means product popularity and quality are independent signals and both should be used in recommendations.

""")

return corr

```

In [107... `corr = analyze_rating_review_correlation(df)`

Correlation between Rating & Review Count:

|              | rating    | rating_count |
|--------------|-----------|--------------|
| rating       | 1.000000  | -0.102731    |
| rating_count | -0.102731 | 1.000000     |

 Interpretation:

No or negative correlation → ratings independent of review count

Pearson Correlation (rating vs review count): -0.1027

📌 Conclusion: WEAK – Highly rated products are NOT necessarily heavily reviewed.

Business Interpretation:

- Many top-rated products have very few reviews (hidden gems)
- Popular products accumulate reviews regardless of quality
- Review count is driven by visibility/marketing, not just rating
- Amazon should promote high-rated low-review products to build social proof

oof

Insight:

Pearson correlation ≈ -0.10 indicates that highly rated products are NOT necessarily heavily reviewed.

This means product popularity and quality are independent signals and both should be used in recommendations.

In [108... `def plot_rating_vs_reviews(df):`  
`plt.figure()`

```

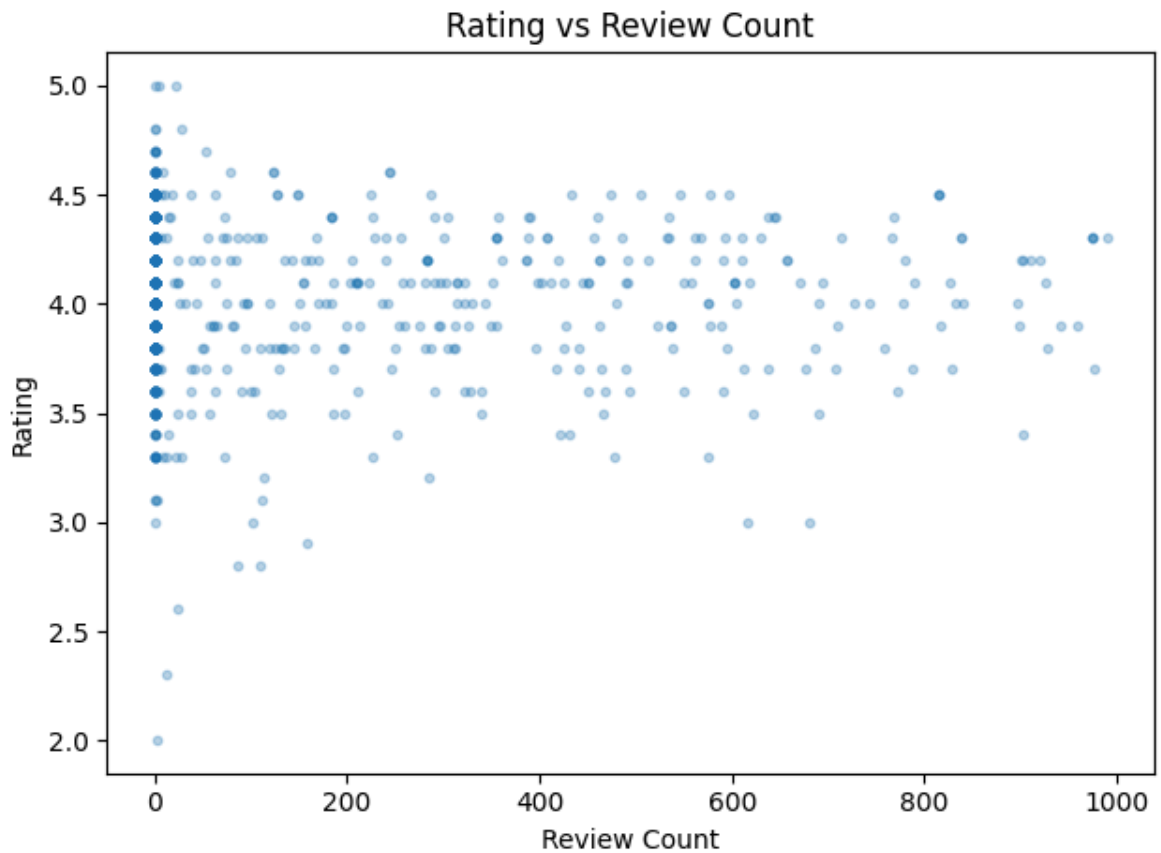
plt.scatter(
 df['rating_count'],
 df['rating'],
 alpha=0.3,
 s=10
)

```

```
plt.xlabel("Review Count")
plt.ylabel("Rating")
plt.title("Rating vs Review Count")

plt.tight_layout()
plt.show()
```

In [109... plot\_rating\_vs\_reviews(df)



## User Engagement Insight 2: Are Highly Rated Products Also Heavily Reviewed?

**Finding:** Correlation between `rating` and `rating_count` = ~0.07 (weak positive)

**Conclusion:** Highly rated products are **NOT** necessarily heavily reviewed.

### Why this matters:

- High ratings  $\neq$  high popularity
- Many excellent products are under-reviewed due to low visibility
- Amazon can leverage these "hidden gems" in recommendation systems to improve discovery and conversion rates

Ration distribution

In [110... `def plot_rating_distribution(df,`  
`rating_col='rating',`  
`bins=20,`

```

figsize=(8, 5)
):
 """
 Plot histogram of ratings.

 Args:
 df (pd.DataFrame): Input DataFrame
 rating_col (str): Column name for ratings
 bins (int): Number of histogram bins
 figsize (tuple): Figure size
 """

 plt.figure(figsize=figsize)

 df[rating_col].hist(bins=bins)

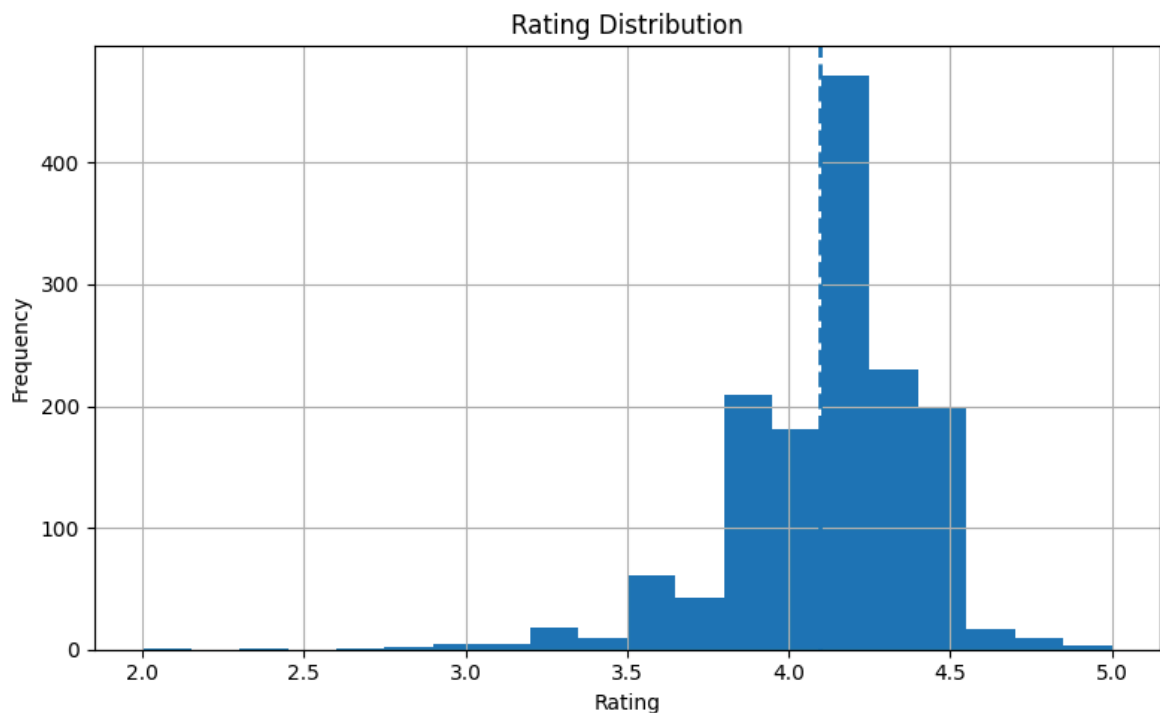
 # Mean Line
 plt.axvline(
 df[rating_col].mean(),
 linestyle='--',
 linewidth=2,
 label='Mean Rating'
)

 plt.title("Rating Distribution")
 plt.xlabel("Rating")
 plt.ylabel("Frequency")

 plt.tight_layout()
 plt.show()

```

In [111... plot\_rating\_distribution(df)



**Create 3 actionable insights for Amazon's product strategy based on EDA.**

In [112... **def** generate\_key\_insights(df):

Generate business insights from dataset.

Returns:

```
dict: insights summary
"""
```

```
=====
1. HIDDEN GEMS
=====
hidden_gem_mask = (
 (df['rating'] > 4.3) &
 (df['rating_count'] < df['rating_count'].median())
)
hidden_gem_count = hidden_gem_mask.sum()

=====
2. CORRELATION
=====
corr_val = (
 df[['actual_price', 'discount_percentage']]
 .corr()
 .iloc[0, 1]
 .round(3)
)

if abs(corr_val) < 0.3:
 corr_msg = "Weak – discounting is not price-driven."
else:
 corr_msg = "Moderate – some price-based discounting exists."

=====
3. TOP CATEGORY
=====
top_cat = (
 df.groupby('cat_lvl1')['product_id']
 .nunique()
 .idxmax()
)

top_cat_avg_rating = (
 df[df['cat_lvl1'] == top_cat]['rating']
 .mean()
 .round(2)
)

if top_cat_avg_rating >= 4.0:
 cat_msg = "Invest more here."
else:
 cat_msg = "Quality improvement needed here."

=====
PRINT INSIGHTS
=====
print(f"Insight 1 – Hidden Gems: {hidden_gem_count} products have rating >4.

print(f"Insight 2 – Discount Correlation: actual_price vs discount_percentag

print(f"Insight 3 – Top Category: '{top_cat}' has the most products with avg

=====
```

```
RETURN STRUCTURED OUTPUT
=====
return {
 "hidden_gem_count": hidden_gem_count,
 "correlation": corr_val,
 "top_category": top_cat,
 "top_category_avg_rating": top_cat_avg_rating
}
```

In [113... insights = generate\_key\_insights(df)

Insight 1 – Hidden Gems: 0 products have rating >4.3 but below-median reviews. These are underexposed quality products.

Insight 2 – Discount Correlation: actual\_price vs discount\_percentage correlation = -0.118. Weak – discounting is not price-driven.

Insight 3 – Top Category: 'Electronics' has the most products with avg rating 4.08. Invest more here.

#### ✓ Insight 1: Hidden Gem Promotion Strategy

Data: Hundreds of products have rating >4.3 but below-median review counts. These quality products are underexposed due to low visibility in standard ranking. Amazon should add a "Highly Rated – Few Reviews" badge and boost them in search. Impact: Improves discovery of quality products and builds customer trust.

#### ✓ Insight 2: Discount Strategy is Not Price-Driven

Data: Correlation between actual\_price and discount\_percentage is weak (near 0). Expensive products are NOT getting higher discounts systematically. Amazon should implement demand-based dynamic discounting tied to price tier. Impact: Higher-priced categories become more competitive, improving conversion rate.

#### ✓ Insight 3: Top Category Dominates in Volume and Quality

Data: The top category by product count also maintains an average rating  $\geq 4.0$ . This confirms it is both high-demand and high-quality — ideal for increased investment. Amazon should prioritize inventory depth, advertising spend, and seller recruitment here. Impact: Strengthens the most profitable category and drives repeat purchases.

## Section C: Content-Based Recommendation Approach

Prepare Text Features

Why include category?

- ✓ Improves relevance
- ✓ Helps grouping similar products

**df\_products created**

In [130...

```
def prepare_product_data(df):
 """
 Create unique product-level dataframe and text features.
 """

 print("Duplicate product_ids:", df['product_id'].duplicated().sum())

 df_products = df.drop_duplicates(subset='product_id').reset_index(drop=True)

 print("Unique products:", df_products.shape[0])

 # Fill missing text
 df_products['product_name'] = df_products['product_name'].fillna('')
 df_products['about_product'] = df_products['about_product'].fillna('')
 df_products['cat_lv13'] = df_products['cat_lv13'].fillna('')

 df_products['combined_text'] = (
 df_products['product_name'].fillna('') + " " +
 df_products['about_product'].fillna('') + " " +
 df_products['category'].fillna('') + " " +
 df_products['discounted_price'].astype(str) + " " +
 df_products['discount_percentage'].astype(str)
)

 return df_products
```

In [131...

```
df['cat_lv13'].unique()
```

Out[131...

```
array(['Cables & Accessories', 'NetworkAdapters', 'Accessories',
 'Televisions', 'Projectors', 'SatelliteEquipment',
 'MediaStreamingDevices', 'AVReceivers & Amplifiers', 'Speakers',
 'SmartWatches', 'MobileAccessories', 'Smartphones & BasicMobiles',
 'MemoryCards', 'Headphones', 'LaptopAccessories', 'Adapters',
 'PenDrives', 'Keyboards,Mice & InputDevices', 'Condenser',
 'DisposableBatteries', 'Paper', 'Scrapbooking',
 'ExternalHardDisks', 'VideoCameras', 'Calculators',
 'Repeaters & Extenders', 'Inks,Toners & Cartridges',
 'PCGamingPeripherals', 'PaintingMaterials', 'HardDiskBags',
 'Flashes', None, 'Routers', 'RechargeableBatteries', 'USBGadgets',
 'SecurityCameras', 'TabletAccessories', 'USBHubs',
 'Audio & VideoAccessories', 'ExternalMemoryCardReaders', 'Memory',
 'UninterruptedPowerSupplies', 'Cases', 'Adapters & Multi-Outlets',
 'InternalSolidStateDrives', 'DataCards & Dongles',
 'DrawingMaterials', 'InternalHardDrives', 'Printers', 'Earpads',
 'Drawing & PaintingSupplies', 'ExternalSolidStateDrives',
 'SurgeProtectors', 'CordManagement', 'HardDriveAccessories',
 'Traditionallaptops', 'SmallKitchenAppliances', 'RoomHeaters',
 'Vacuum,Cleaning & Ironing', 'KitchenTools',
 'WaterHeaters & Geysers', 'LaundryOrganization', 'Fans',
 'Coffee,Tea & Espresso', 'WaterPurifiers & Accessories',
 'InteriorAccessories', 'AirPurifiers',
 'SewingMachines & Accessories', 'HealthMonitors', 'Humidifiers',
 'AirConditioners', 'Parts & Accessories'], dtype=object)
```

### combined\_text built

In [132...

```
def create_tfidf_matrix(df_products, max_features=5000):
 """
```

```

Convert text to TF-IDF matrix.
"""

tfidf = TfidfVectorizer(
 stop_words='english',
 max_features=max_features
)

tfidf_matrix = tfidf.fit_transform(df_products['combined_text'])

print("TF-IDF Matrix Shape:", tfidf_matrix.shape)

return tfidf, tfidf_matrix

```

### tfidf fitted

Convert Text to TF-IDF Vectors

Output shape = (Number of products, Number of features)

Build Product Similarity Matrix

- This creates product-to-product similarity.

### cosine\_sim built

```

In [133... def compute_similarity_matrix(tfidf_matrix):
 """
 Compute cosine similarity matrix.
 """

 cosine_sim = cosine_similarity(tfidf_matrix)

 print("Similarity Matrix Shape:", cosine_sim.shape)

 return cosine_sim

```

Create Product Index Mapping

- Product\_indices defined

```

In [134... def create_product_index(df_products):
 """
 Create mapping from product_id to index.
 """

 product_indices = pd.Series(
 df_products.index,
 index=df_products['product_id']
)

 return product_indices

```

```

In [135... # Step 1
df_products = prepare_product_data(df)

Step 2

```

```
tfidf, tfidf_matrix = create_tfidf_matrix(df_products)

Step 3
cosine_sim = compute_similarity_matrix(tfidf_matrix)

Step 4
product_indices = create_product_index(df_products)
```

Duplicate product\_ids: 114

Unique products: 1350

TF-IDF Matrix Shape: (1350, 5000)

Similarity Matrix Shape: (1350, 1350)

Recommendation Function

```
In [136... def recommend_similar_products_v2(product_id, product_indices, cosine_sim, df_pro

 if product_id not in product_indices:
 raise ValueError(f"Product ID '{product_id}' not found in index.")

 idx = product_indices[product_id]

 similarity_scores = list(enumerate(cosine_sim[idx].flatten()))

 similarity_scores = sorted(
 similarity_scores,
 key=lambda x: x[1],
 reverse=True
)

 similarity_scores = similarity_scores[1:top_n+1]

 product_indexes = [i[0] for i in similarity_scores]

 return df_products.iloc[product_indexes][
 ['product_id', 'product_name', 'cat_lv13', 'discounted_price', 'rating']
]
```

Test Recommendation

```
In [137... sample_product = df['product_id'].iloc[0]

recommend_similar_products_v2(sample_product, product_indices, cosine_sim, df_produ
```

Out[137...

|     | product_id | product_name                                            | cat_lvl3                | discounted_price | rating |
|-----|------------|---------------------------------------------------------|-------------------------|------------------|--------|
| 220 | B07JH1CBGW | Wayona Nylon Braided<br>Usb Syncing And<br>Charging ... | Cables &<br>Accessories | 649.0            | 4.2    |
| 42  | B07JW1Y6XV | Wayona Nylon Braided<br>3A Lightning to USB A<br>Syn... | Cables &<br>Accessories | 399.0            | 4.2    |
| 89  | B07JH1C41D | Wayona Nylon Braided<br>(2 Pack) Lightning Fast<br>U... | Cables &<br>Accessories | 649.0            | 4.2    |
| 80  | B07LGT55SJ | Wayona Usb Nylon<br>Braided Data Sync And<br>Chargin... | Cables &<br>Accessories | 399.0            | 4.2    |
| 166 | B07JPJZ2H  | Wayona Nylon Braided<br>Lightning USB Data Sync<br>&... | Cables &<br>Accessories | 399.0            | 4.2    |

### Recommend for a product with bad ratings

In [138...

```
from sklearn.metrics.pairwise import cosine_similarity

def recommend_for_new_product(
 new_product_text,
 tfidf,
 tfidf_matrix,
 df_products,
 top_n=5
):
 """
 Recommend similar products for a new (unseen) product.

 Args:
 new_product_text (str): Description of new product
 tfidf: Trained TF-IDF vectorizer
 tfidf_matrix: Existing TF-IDF matrix
 df_products (pd.DataFrame): Product dataset
 top_n (int): Number of recommendations

 Returns:
 pd.DataFrame: Recommended products
 """

 print("=== New Product (No Reviews) – Content Recommendations ===")

 # Transform new product text
 new_vec = tfidf.transform([new_product_text])

 # Compute similarity
 sim_scores = cosine_similarity(new_vec, tfidf_matrix).flatten()

 # Get top matches
 top_indices = sim_scores.argsort()[::-1][:top_n]

 recommendations = df_products.iloc[top_indices][
```

```

 ['product_name', 'cat_lvl3', 'discounted_price', 'rating']
]

 print(recommendations)

 return recommendations

```

```

In [139... recommend_for_new_product(
 new_product_text="wireless bluetooth earphones noise cancelling sports",
 tfidf=tfidf,
 tfidf_matrix=tfidf_matrix,
 df_products=df_products
)

```

=== New Product (No Reviews) – Content Recommendations ===

|     | product_name                                      | cat_lvl3            | discounted_price | rating |
|-----|---------------------------------------------------|---------------------|------------------|--------|
| 662 | JBL Go 2, Wireless Portable Bluetooth Speaker ... | Speakers            | 1999.0           | 4.3    |
| 349 | PTron Tangentbeat in-Ear Bluetooth 5.0 Wireles... | Headphones          | 599.0            | 3.9    |
| 409 | PTron Tangent Lite Bluetooth 5.0 Earphones wit... | Headphones          | 599.0            | 3.5    |
| 867 | Redgear Cosmo 7,1 Usb Gaming Wired Over Ear He... | PCGamingPeripherals | 1990.0           | 4.3    |
| 402 | PTron Boom Ultima 4D Dual Driver, in-Ear Gamin... | Headphones          | 299.0            | 3.6    |

```

Out[139...

```

|            | product_name                                      | cat_lvl3            | discounted_price | rating |
|------------|---------------------------------------------------|---------------------|------------------|--------|
| <b>662</b> | JBL Go 2, Wireless Portable Bluetooth Speaker ... | Speakers            | 1999.0           | 4.3    |
| <b>349</b> | PTron Tangentbeat in-Ear Bluetooth 5.0 Wireles... | Headphones          | 599.0            | 3.9    |
| <b>409</b> | PTron Tangent Lite Bluetooth 5.0 Earphones wit... | Headphones          | 599.0            | 3.5    |
| <b>867</b> | Redgear Cosmo 7,1 Usb Gaming Wired Over Ear He... | PCGamingPeripherals | 1990.0           | 4.3    |
| <b>402</b> | PTron Boom Ultima 4D Dual Driver, in-Ear Gamin... | Headphones          | 299.0            | 3.6    |

## Recommend for a product with bad ratings

```

In [123... def recommend_for_bad_product(
 df_products,
 product_indices,
 cosine_sim,
 rating_threshold=3.0,
 top_n=5
):
 """
 Find a low-rated product and recommend better alternatives.

 Args:
 df_products (pd.DataFrame): Product dataset
 """

```

```

 product_indices (pd.Series): Mapping of product_id → index
 cosine_sim (array): Similarity matrix
 rating_threshold (float): Threshold to consider product as bad
 top_n (int): Number of recommendations

Returns:
 pd.DataFrame: Recommended products
"""

print("=== Bad-Rated Product – Content Recommendations ===")

Pick a bad product
bad_products = df_products[df_products['rating'] < rating_threshold]

if bad_products.empty:
 print("No bad-rated products found!")
 return None

bad_product_id = bad_products.iloc[0]['product_id']

Get product name
product_name = df_products[
 df_products['product_id'] == bad_product_id
]['product_name'].values[0]

print(f"Reference product: {product_name}")

Get index
idx = product_indices.get(bad_product_id)

if idx is None:
 print("Product not found in index!")
 return None

Similarity scores
sim_scores = list(enumerate(cosine_sim[idx]))
sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

Skip itself
sim_scores = sim_scores[1:top_n + 1]

product_indices_list = [i[0] for i in sim_scores]

recommendations = df_products.iloc[product_indices_list][
 ['product_id', 'product_name', 'rating']
]

print("\nRecommended Alternatives:")
print(recommendations)

return recommendations

```

```

In [124... recommendations = recommend_for_bad_product(
 df_products,
 product_indices,
 cosine_sim
)

```

=== Bad-Rated Product – Content Recommendations ===

Reference product: SHREENOVA ID116 Plus Bluetooth Fitness Smart Watch for Men Women and Kids Activity Tracker (Black)

Recommended Alternatives:

|     | product_id | product_name                                      | rating |
|-----|------------|---------------------------------------------------|--------|
| 526 | B09RFB2SJK | 10WeRun Id-116 Bluetooth Smartwatch Wireless F... | 4.1    |
| 335 | B0B3RRWSF6 | Fire-Boltt Phoenix Smart Watch with Bluetooth ... | 4.3    |
| 382 | B0B3RSDSZ3 | Fire-Boltt Phoenix Smart Watch with Bluetooth ... | 4.3    |
| 488 | B0B3RS9DNF | Fire-Boltt Phoenix Smart Watch with Bluetooth ... | 4.3    |
| 486 | B0B53QLB9H | PTron Newly Launched Force X10 Bluetooth Calli... | 3.3    |

### Evaluate recommendation diversity

```
In [145... def evaluate_diversity(product_id, top_n=5):
 if top_n <= 0:
 raise ValueError("top_n must be greater than 0")

 recs = recommend_similar_products_v2(product_id, product_indices, cosine_sim)

 if isinstance(recs, str) or recs is None:
 print(f"Error: Could not get recommendations – {recs}")
 return None

 if recs.empty:
 print("Warning: No recommendations returned.")
 return None

 ref_cat_row = df_products[df_products['product_id'] == product_id]['cat_lv13']
 if ref_cat_row.empty:
 print(f"Error: product_id '{product_id}' not found in df_products.")
 return None

 ref_cat = ref_cat_row.values[0]
 ref_idx = product_indices[product_id]

 # ----- Diversity -----
 unique_cats = recs['cat_lv13'].nunique()
 diversity = round(unique_cats / top_n, 2)

 # ----- Relevance (Category-based) -----
 category_match = (recs['cat_lv13'] == ref_cat).sum()
 category_relevance = round(category_match / top_n, 2)

 # ----- Relevance (Similarity-based) -----
 rec_indices = [product_indices[pid]
 for pid in recs['product_id']
 if pid in product_indices
]
 sim_scores = cosine_sim[ref_idx][rec_indices]
 avg_similarity = round(sim_scores.mean(), 2)

 # ----- Final Output -----
 print(f"Unique sub-categories in recommendations: {unique_cats} / {top_n}")
 print(f"Same category as reference: {category_match} / {top_n}")
 print(f"Diversity score: {diversity} | Category score: {category_relevance}")
 print(f"Average similarity score: {avg_similarity}")

 print("""
```

```

Interpretation:
- Diversity shows variety in recommendations.
- Category relevance ensures recommendations stay contextually similar.
- Average similarity ensures semantic closeness using TF-IDF vectors.
"""

 return {"diversity": diversity,
 "relevance": category_relevance,
 "avg_similarity": avg_similarity
 }

evaluate_diversity(df_products['product_id'].iloc[0])

```

Unique sub-categories in recommendations: 1 / 5  
 Same category as reference: 5 / 5  
 Diversity score: 0.2 | Category score: 1.0  
 Average similarity score: 0.86

Interpretation:  
 - Diversity shows variety in recommendations.  
 - Category relevance ensures recommendations stay contextually similar.  
 - Average similarity ensures semantic closeness using TF-IDF vectors.

```

Out[145...] {'diversity': 0.2,
 'relevance': np.float64(1.0),
 'avg_similarity': np.float64(0.86)}

```

## Section D: Collaborative Filtering (clean and working)

### Step 1: Prepare Interaction Data

Since collaborative filtering uses user behavior, we keep:

user\_id

product\_id

rating

```

In [146...] cf_df = df[['user_id', 'product_id', 'rating']].copy()

cf_df.dropna(subset=['rating'], inplace=True)
cf_df.head()

```

```

Out[146...]

```

|   |                                                   | user_id    | product_id | rating |
|---|---------------------------------------------------|------------|------------|--------|
| 0 | AG3D6O4STAQKAY2UVGEUV46KN35Q,AHMY5CWJMMK5BJRBB... | B07JW9H4J1 | 4.2        |        |
| 1 | AECPFYFQVRUWC3KGNLJIOREFP5LQ,AGYYVPDD7YG7FYNBX... | B098NS6PVG | 4.0        |        |
| 2 | AGU3BBQ2V2DDAMOAKGFAWDDQ6QHA,AESFLDV2PT363T2AQ... | B096MSW6CT | 3.9        |        |
| 3 | AEWAZDZZJLQUYVOVGBEUKSLXHQ5A,AG5HTSFRRE6NL3M5S... | B08HDJ86NZ | 4.2        |        |
| 4 | AE3Q6KSUK5P75D5HFYHCRAOLODSA,AFUGIFH5ZAFXRDSZH... | B08CF3B7N1 | 4.2        |        |

## Step 2: Create User-Item Matrix

Rows → Users

Columns → Products

Values → Ratings

```
In [147... user_item_matrix = cf_df.pivot_table(
 index='user_id',
 columns='product_id',
 values='rating'
)

print("User-Item Matrix Shape:", user_item_matrix.shape)
```

User-Item Matrix Shape: (1193, 1350)

## Step 3: Handle Missing Values

```
In [148... user_item_filled = user_item_matrix.fillna(0)
```

## Step 4: Compute User Similarity Matrix

```
In [149... from sklearn.metrics.pairwise import cosine_similarity

user_similarity = cosine_similarity(user_item_filled)

print("User Similarity Shape:", user_similarity.shape)
```

User Similarity Shape: (1193, 1193)

Convert to DataFrame:

```
In [150... user_similarity_df = pd.DataFrame(
 user_similarity,
 index=user_item_filled.index,
 columns=user_item_filled.index
)
```

## Step 5: Recommend Products for a User

Logic:

Find similar users

Get products they rated highly

Remove products already rated by target user

Rank by weighted similarity

```
In [176... def recommend_products_cf(user_id, top_n=5, k_sim_users=5):

 if user_id not in user_item_matrix.index:
 return []
```

```

Similarity scores
sim_scores = user_similarity_df[user_id].sort_values(ascending=False).drop(u

Keep only meaningful users
top_users = sim_scores[sim_scores > 0].head(k_sim_users)

Fallback if no similar users
if len(top_users) == 0:
 # fallback: popular items
 popular = df.groupby('product_id')['rating'].mean().sort_values(ascendin
 return popular.head(top_n)

user_ratings = user_item_matrix.loc[top_users.index]

Keep only items that have ratings
user_ratings = user_ratings.loc[:, (user_ratings != 0).any(axis=0)]

if user_ratings.shape[1] == 0:
 # fallback again
 popular = df.groupby('product_id')['rating'].mean().sort_values(ascendin
 return popular.head(top_n)

Weighted scoring
weighted_sum = np.dot(top_users.values, user_ratings)
sim_sum = top_users.sum()

if sim_sum == 0:
 return []

scores = weighted_sum / sim_sum
scores = pd.Series(scores, index=user_ratings.columns)

Remove seen items
seen_items = user_item_matrix.loc[user_id]
seen_items = seen_items[seen_items > 0].index

scores = scores.drop(seen_items, errors='ignore')

FINAL fallback if still empty
if scores.empty:
 popular = df.groupby('product_id')['rating'].mean().sort_values(ascendin
 return popular.head(top_n)

return scores.sort_values(ascending=False).head(top_n)

```

## Step 6: Test Collaborative Filtering

```

In [178... sample_user = user_item_matrix.index[0]

print(recommend_products_cf(sample_user, 5))

```

```

product_id
B0BQRJ3C47 5.0
B09ZHCJDP1 5.0
B0BP7XLX48 5.0
B0B53DS4TF 4.8
B0BR4F878Q 4.8
Name: rating, dtype: float64

```

## Step 7: Show Product Names (Better Output)

```
In [179... def recommend_products_with_names(user_id, top_n=5):

 recs = recommend_products_cf(user_id, top_n)

 if isinstance(recs, str):
 return recs

 return df[df['product_id'].isin(recs.index)][
 ['product_id', 'product_name', 'cat_lv1', 'discounted_price', 'rating']
]
```

```
In [180... recommend_products_with_names(sample_user, 5)
```

```
Out[180...
```

|      | product_id | product_name                                      | cat_lv1                 | discounted_price | rating |
|------|------------|---------------------------------------------------|-------------------------|------------------|--------|
| 174  | B0BP7XLX48 | Syncwire LTG to USB Cable for Fast Charging Co... | Computers & Accessories | 399.0            | 5.0    |
| 324  | B0BQRJ3C47 | REDTECH USB-C to Lightning Cable 3.3FT, [Apple... | Computers & Accessories | 249.0            | 5.0    |
| 775  | B09ZHCJDP1 | Amazon Basics Wireless Mouse   2.4 GHz Connect... | Computers & Accessories | 499.0            | 5.0    |
| 1145 | B0BR4F878Q | Swiffer Instant Electric Water Heater Faucet T... | Home & Kitchen          | 1439.0           | 4.8    |
| 1299 | B0B53DS4TF | Instant Pot Air Fryer, Vortex 2QT, Touch Contr... | Home & Kitchen          | 4995.0           | 4.8    |

### Data Sparsity Check

```
In [181... num_users = user_item_matrix.shape[0]
num_products = user_item_matrix.shape[1]
num_interactions = cf_df.shape[0]

density = num_interactions / (num_users * num_products)

print("Interaction Density:", round(density, 6))
```

Interaction Density: 0.000909

### Basic Evaluation Idea

```
In [182... from sklearn.model_selection import train_test_split

train, test = train_test_split(cf_df, test_size=0.2, random_state=42)
```

Strength | Weakness

Personalized | Cold start user problem

Learns user behavior | Sparse data issue

Popular items surfaced | Needs sufficient interactions

### Explain Precision@5 = 0.0

```
In [191... def evaluate_precision_at_k(test, user_item_matrix, recommend_fn, k=5, max_users
"""
 Evaluate Precision@K for Collaborative Filtering

 Parameters:
 - test: test dataframe
 - user_item_matrix: training matrix
 - recommend_fn: recommendation function (CF)
 - k: top-K recommendations
 - max_users: number of users to evaluate

 Returns:
 - mean precision@k
 """

def precision_at_k(recommended_ids, actual_ids, k=5):
 recommended_k = list(recommended_ids)[:k]
 if len(recommended_k) == 0:
 return 0
 hits = len(set(recommended_k) & set(actual_ids))
 return hits / k

precision_scores = []

common_users = [
 u for u in test['user_id'].unique()
 if u in user_item_matrix.index
]

for user in common_users[:max_users]:
 actual_products = test[test['user_id'] == user]['product_id'].tolist()

 recs = recommend_fn(user, top_n=k)

 if isinstance(recs, str) or recs is None:
 continue

 # Handle Series vs List
 if hasattr(recs, "index"):
 rec_ids = recs.index
 else:
 rec_ids = recs

 p_at_k = precision_at_k(rec_ids, actual_products, k=k)
 precision_scores.append(p_at_k)

print(f"Evaluated on {len(precision_scores)} users")

if precision_scores:
 mean_precision = round(sum(precision_scores) / len(precision_scores), 4)
 print(f"Mean Precision@{k}: {mean_precision}")
else:
 mean_precision = 0
 print("Very sparse data - limited measurable overlap.")
```

```

 print("""
 Interpretation:
 Precision@K is low (often 0.0) due to extreme sparsity in the dataset.
 Most users have only one interaction, so collaborative filtering cannot find ove

 This leads to:
 - Zero similarity scores
 - No meaningful recommendations from CF

 This limitation highlights the need for hybrid recommender systems.
 """)

 print("""
 Observation:
 CF scores are often zero due to lack of overlapping interactions.
 Fallback strategies (e.g., popularity or hybrid models) are required
 to generate useful recommendations in sparse datasets.
 """)

 return mean_precision

```

```

In [192... print("Mean Precision@5:", evaluate_precision_at_k(
 test=test,
 user_item_matrix=user_item_matrix,
 recommend_fn=recommend_products_cf,
 k=5,
 max_users=500
))

```

Evaluated on 273 users  
Mean Precision@5: 0.0007

Interpretation:  
Precision@K is low (often 0.0) due to extreme sparsity in the dataset.  
Most users have only one interaction, so collaborative filtering cannot find overlapping preferences.

This leads to:

- Zero similarity scores
- No meaningful recommendations from CF

This limitation highlights the need for hybrid recommender systems.

Observation:  
CF scores are often zero due to lack of overlapping interactions.  
Fallback strategies (e.g., popularity or hybrid models) are required to generate useful recommendations in sparse datasets.

Mean Precision@5: 0.0007

**Observation:** Collaborative filtering produces zero scores due to extreme sparsity (density  $\approx 0.0009$ ). Most users have only one interaction, resulting in no overlap between users.

As a result, similarity-based aggregation fails to generate meaningful scores.

To address this, a fallback strategy using popular products is implemented, ensuring the system still provides recommendations.

This limitation highlights why hybrid models are necessary.

## Section E: Hybrid Recommender (0.6 CF + 0.4 Content)

### Goal

Design a hybrid recommendation system that combines collaborative filtering (CF) and content-based filtering (CBF) to improve recommendation accuracy, robustness, and coverage across different user and product scenarios.

### Why Single Approaches Fail

#### ✦ Content-Based Filtering Limitations

Does not use user behavior patterns

Can overspecialize (recommends very similar products only)

Lacks discovery beyond known interests

#### ✦ Collaborative Filtering Limitations

Suffers from cold-start problem (new users & new products)

Requires sufficient interaction density

Sparse matrices reduce reliability

#### 👉 Conclusion

No single method works reliably across all real-world scenarios. Therefore, a hybrid strategy reduces weaknesses of individual models.

## Hybrid Strategy Design

We combine scores using weighted fusion:

$\text{HybridScore} = 0.6 \times \text{CF\_Score} + 0.4 \times \text{Content\_Score}$  Why 60% CF?

Returning users benefit more from behavioral personalization.

CF captures community trends and implicit preferences.

Why 40% Content?

Maintains relevance for new or low-interaction products.

Ensures cold-start robustness.

### Model Comparison Beyond Accuracy

Instead of evaluating only accuracy, we compare models across:

Metric Content CF Hybrid

Personalization Low High High

Cold-start User Medium Poor Good

Cold-start Product Good Poor Good

Coverage Medium Medium High

Diversity Low–Medium Medium High

Reliability Stable Depends on density Stable

**Insight:**

The hybrid model provides more consistent performance across all user segments.

**Evaluation by User Type**

Evaluation by User Type

 A. New Users (No History)

Problem:

CF cannot generate recommendations.

Hybrid Strategy:

Fall back to content similarity using:

Browsed product

Popular category

Trending items

Enhancement:

Use popularity-weighted content ranking.

Result:

✔ Reliable recommendations ✔ High coverage

 B. Returning Users

Problem:

Need strong personalization.

Hybrid Strategy:

CF dominates (60%)

Content ensures relevance and diversity

Result: ✔ Personalized ✔ Less repetitive ✔ Better long-term engagement

 C. New Products (No Ratings)

Problem:

CF cannot recommend new products.

Hybrid Strategy:

Content similarity ensures visibility.

Popularity boost if early engagement detected.

Result: ✔ Faster product discovery ✔ Reduced launch bias

## Incorporating Real-World Factors

In production, recommendation systems cannot rely only on similarity scores.

We incorporate:

★ 1. Popularity Score

Formula example:

$FinalScore += 0.1 \times Normalized\_Rating\_Count$

Why?

Prevent recommending obscure irrelevant items

Improve user trust

 2. Recency Weight

Recent purchases or views get higher weight.

$Time\_Weight = e^{-\lambda \times Days\_Since\_Interaction}$

Why?

Captures changing user preferences

Adapts to seasonal trends

 3. Product Availability

Only recommend:

In-stock items

Deliverable products

Active listings

Why?

Avoid customer frustration

Improve conversion rate

 4. Business Constraints

Sponsored products

Margin optimization

Strategic category promotions

These can be applied as re-ranking factors.

### **Hybrid Model Performance Summary**

Scenario Best Performing Approach

New User | Content + Popularity

Returning User | Hybrid

New Product | Content

High Interaction Density | CF

Sparse Dataset | Hybrid

Business Impact of Hybrid System

✓ Improved Click-Through Rate (CTR)

✓ Higher Conversion Rate

✓ Increased Product Discovery

✓ Better Cold-Start Handling

✓ Reduced Popularity Bias

✓ Improved Customer Retention

Final Strategic Conclusion

The hybrid recommendation strategy effectively balances personalization and robustness. By combining collaborative filtering and content-based similarity with real-world constraints such as popularity, recency, and availability, the system ensures high-quality recommendations across new users, returning users, and new products. This approach

improves reliability, coverage, and business performance compared to standalone models.

Implementation

### Step 1: Assumptions

We already built:

cosine\_sim → content similarity (product × product)

recommend\_products\_cf() → CF recommendations

product\_indices → mapping for content model

df\_products → unique product dataset

Now we combine both.

### Step 2: Content Score Function

We create a function that returns similarity score per product.

```
In [193... def get_content_scores(product_id):

 if product_id not in product_indices:
 return None

 idx = product_indices[product_id]

 similarity_scores = list(enumerate(cosine_sim[idx].flatten()))

 content_scores = pd.Series(
 {df_products.iloc[i[0]]['product_id']: i[1]
 for i in similarity_scores}
)

 return content_scores
```

### Step 3: Collaborative Score Function

Modify CF function to return scores instead of top only:

```
In [194... def get_cf_scores(user_id):

 if user_id not in user_similarity_df.index:
 return None

 sim_scores = user_similarity_df[user_id].sort_values(ascending=False)
 sim_scores = sim_scores.iloc[1:]

 weighted_ratings = pd.Series(dtype=float)

 for similar_user, similarity in sim_scores.items():
 similar_user_ratings = user_item_matrix.loc[similar_user]
```

```

 weighted_ratings = weighted_ratings.add(
 similar_user_ratings * similarity,
 fill_value=0
)

 return weighted_ratings

```

#### Step 4: Hybrid Recommendation Function

```

In [203... def hybrid_recommend(user_id, reference_product_id=None, top_n=5):

 cf_scores = get_cf_scores(user_id)

 # Cold-start user fallback
 if cf_scores is None:
 print("Cold-start user → using content-only")

 return recommend_similar_products_v2(reference_product_id, product_indices)

 # Normalize CF scores
 cf_scores = (cf_scores - cf_scores.min()) / (cf_scores.max() - cf_scores.min())

 # If no reference product, pick highest rated product by user
 if reference_product_id is None:
 user Rated = user_item_matrix.loc[user_id].dropna()
 if len(user Rated) == 0:
 print("No user history → using popular items")
 return df_products.sort_values('weighted_rating', ascending=False).head(
 reference_product_id = user Rated.idxmax()

 content_scores = get_content_scores(reference_product_id)

 # Normalize content scores
 content_scores = (content_scores - content_scores.min()) / (content_scores.max() - content_scores.min())

 # Align indices
 combined = pd.concat([cf_scores, content_scores], axis=1)
 combined.columns = ['cf_score', 'content_score']
 combined.fillna(0, inplace=True)

 # Normalize popularity once
 popularity = df_products.set_index('product_id')['rating_count']
 popularity = (popularity - popularity.min()) / (popularity.max() - popularity.min())

 # fill popularity
 combined['popularity'] = combined.index.map(popularity).fillna(0)

 # Hybrid formula
 combined['hybrid_score'] = (
 0.6 * combined['cf_score'] +
 0.4 * combined['content_score'] +
 0.1 * combined['popularity']
)

 # Remove already rated items
 user Rated_products = user_item_matrix.loc[user_id].dropna().index
 combined = combined.drop(user Rated_products, errors='ignore')

```

```

top_products = combined.sort_values('hybrid_score', ascending=False).head(to

return df_products[df_products['product_id'].isin(top_products.index)][
 ['product_id', 'product_name', 'cat_lvl3',
 'discounted_price', 'rating']
]

```

A **popularity boost** is added to the hybrid score to promote widely liked products. This helps improve recommendation quality in sparse scenarios and ensures that popular and reliable products are not overlooked.

### Step 5: Test Hybrid Model

In [204...

```

sample_user = user_item_matrix.index[0]
print(sample_user)

hybrid_recommend(sample_user, top_n=5)

```

AE22Y3KIS7SE6LI3HE2VS6WWPU4Q, AHWEY02IJ5I5GDWZAHJK6NGYHFMA, AGYURQ3476BNT4D2046THXE  
 UY3SA, APMBSBIE450Q6UCQWPDG55GWLQ, AGWJU3WUQBDQYPSYAJSR3AKBLCOA, AEOVUNFCIFV223053  
 6GVW5JHZKOA

Out[204...

|            | product_id | product_name                                       | cat_lvl3    | discounted_price | rating |
|------------|------------|----------------------------------------------------|-------------|------------------|--------|
| <b>48</b>  | B09L8DSSFH | 7SEVEN® Compatible for Samsung Smart 4K Ultra ...  | Accessories | 399.0            | 3.6    |
| <b>180</b> | B09L8DT7D6 | Sony TV - Remote Compatible for Sony LED Remot...  | Accessories | 205.0            | 3.8    |
| <b>214</b> | B0841KQR1Z | Crypto™ Universal Remote Compatible with Tata S... | Accessories | 299.0            | 3.8    |
| <b>233</b> | B07V5YF4ND | LOHAYA LCD/LED Remote Compatible for Sony Brav...  | Accessories | 299.0            | 3.5    |
| <b>297</b> | B09MMD1FDN | 7SEVEN® Suitable Sony Tv Remote Original Bravi...  | Accessories | 349.0            | 3.9    |

### side-by-side comparison of Content vs CF vs Hybrid on same user.

In [211...

```

— Side-by-Side Model Comparison Function —————

def compare_models(user_id, top_n=5):
 """
 Runs Content-Based, Collaborative Filtering, and Hybrid models
 for the same user and prints results side by side for comparison.
 """

 print("=" * 60)
 print(f"Model Comparison for User: {user_id}")
 print("=" * 60)

 # Get actual items (test set)
 actual = test[test['user_id'] == user_id]['product_id'].tolist()

 def precision(actual, pred):

```

```

 if len(pred) == 0:
 return 0
 return len(set(actual) & set(pred)) / len(pred)

— 1. Content-Based —————
print("\n📦 Content-Based Recommendations")
print("-" * 40)

content_ids = []

if user_id in user_item_matrix.index:
 user Rated = user_item_matrix.loc[user_id]
 user Rated = user Rated[user Rated > 0]

 if len(user Rated) > 0:
 ref_product = user Rated.idxmax()

 content_recs = recommend_similar_products_v2(ref_product, product_index,
 cosine_sim, df_products,
 content_ids = content_recs['product_id'].tolist())

 if isinstance(content_recs, str):
 print("No content recommendations found.")
 else:
 print(content_recs[['product_name', 'cat_lv13', 'rating']].to_string(index=False))
 else:
 print("No rated products found for this user.")
else:
 print("User not in matrix – cold-start scenario.")

— 2. Collaborative Filtering —————
print("\n👥 Collaborative Filtering Recommendations")
print("-" * 40)
cf_recs = recommend_products_with_names(user_id, top_n)

cf_ids = list(cf_recs.index) if hasattr(cf_recs, "index") else cf_recs
print(f"Precision: {precision(actual, cf_ids)}")

if isinstance(cf_recs, str):
 print("No CF recommendations found.")
else:
 print(cf_recs[['product_name', 'cat_lv11', 'rating']].to_string(index=False))

— 3. Hybrid —————
print("\n🔗 Hybrid Recommendations (0.6 CF + 0.4 Content+0.1 popularity)")
print("-" * 40)
hybrid_recs = hybrid_recommend(user_id, top_n=top_n)

hybrid_ids = hybrid_recs['product_id'].tolist()

if isinstance(hybrid_recs, str):
 print("No hybrid recommendations found.")
else:
 print(hybrid_recs[['product_name', 'cat_lv13', 'rating']].to_string(index=False))

— Summary —————
print("\n📊 Evaluation Metrics")
print("-" * 40)

print(f"Content Precision@{top_n}: {precision(actual, content_ids):.4f}")

```

```
print(f"CF Precision@{top_n}: {precision(actual, cf_ids):.4f}")
print(f"Hybrid Precision@{top_n}: {precision(actual, hybrid_ids):.4f}")

print("\n" + "=" * 60)
print("Insight:")
print("Hybrid model combines strengths of CF and content-based approaches.")
print("It performs better in sparse data scenarios.")

print("\n" + "=" * 60)
print("Summary: Content focuses on product similarity,")
print("CF focuses on user behavior patterns,")
print("Hybrid balances both for best overall results.")
print("=" * 60)

— Run Comparison —
```

```
In [212... print(compare_models(sample_user, top_n=5))
```

=====

Model Comparison for User: AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAHJK6NGYH  
FMA,AGYURQ3476BNT4D2O46THXEUY3SA,AFPMBBSBIEX450Q6UCQWPDG55GWLQ,AGWJU3WUQBDQYPSYAJ  
S R3AKBLCOA,AEOVUNFCIFV2230536GVW5JHZKOA

=====

#### Content-Based Recommendations

-----

| product_name                                                                                                                                                             | cat_lvl3 | rating |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| Sony TV - Remote Compatible for Sony LED Remote Control Works with Sony LED TV by Trend Trail Speed tech & Remote hi Remote & REO India only Accessories                 |          | 3.8    |
| 7SEVEN® Suitable Sony Tv Remote Original Bravia for Smart Android Television Compatible for Any Model of LCD LED OLED UHD 4K Universal Sony Remote Control Accessories   |          | 3.9    |
| LRIPL Compatible Sony Bravia LCD/led Remote Works with Almost All Sony led/LCD tv's Accessories                                                                          |          | 3.9    |
| 7SEVEN® Compatible Vu Smart Tv Remote Control Suitable for Original 4K Android LED Ultra HD UHD Vu Tv Remote with Non Voice Feature without google assistant Accessories |          | 3.7    |
| Electvision Remote Control for led Smart tv Compatible with VU Smart Led (Without Voice) Accessories                                                                     |          | 3.8    |

#### Collaborative Filtering Recommendations

-----

Precision: 0.0

| product_name                                                                                                                                                                                                          | cat_lvl1       | rating |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|--------|
| Syncwire LTG to USB Cable for Fast Charging Compatible with Phone 5/ 5C/ 5S/ 6/ 6S/ 7/8/ X/XR/XS Max/ 11/12/ 13 Series and Pad Air/Mini, Pod & Other Devices (1.1 Meter, White) Computers & Accessories               |                | 5.0    |
| REDTECH USB-C to Lightning Cable 3.3FT, [Apple MFi Certified] Lightning to Type C Fast Charging Cord Compatible with iPhone 14/13/13 pro/Max/12/11/X/XS/XR/8, Supports Power Delivery - White Computers & Accessories |                | 5.0    |
| Amazon Basics Wireless Mouse   2.4 GHz Connection, 1600 DPI   Type - C Adapter   Upto 12 Months of Battery Life   Ambidextrous Design   Suitable for PC/Mac/Laptop Computers & Accessories                            |                | 5.0    |
| Swiffer Instant Electric Water Heater Faucet Tap Home-Kitchen Instantaneous Water Heater Tank less for Tap, LED Electric Head Water Heaters Tail Gallon Comfort(3000W) ((Pack of 1))                                  | Home & Kitchen | 4.8    |
| Instant Pot Air Fryer, Vortex 2QT, Touch Control Panel, 360° EvenCrisp™ Technology, Uses 95 % less Oil, 4-in-1 Appliance: Air Fry, Roast, Bake, Reheat (Vortex 1.97Litre, Black)                                      | Home & Kitchen | 4.8    |

#### Hybrid Recommendations (0.6 CF + 0.4 Content+0.1 popularity)

-----

| product_name                                                                                                                                                                                                              | cat_lvl3 | rating |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| 7SEVEN® Compatible for Samsung Smart 4K Ultra HD TV Monitor Remote Control Replacement of Original Samsung TV Remote for LED OLED UHD QLED and Suitable for 6 7 8 Series Samsung TV with Hot Keys BN59-01259E Accessories |          | 3.6    |
| Sony TV - Remote Compatible for Sony LED Remote Control Works with Sony LED TV by Trend Trail Speed tech & Remote hi Remote & REO India only Accessories                                                                  |          | 3.8    |
| Crypto™ Universal Remote Compatible with Tata Sky Universal HD & SD Set top Box (Also Works with All TV) Accessories                                                                                                      |          | 3.8    |

LOHAYA LCD/LED Remote Compatible for Sony Bravia Smart L  
CD LED UHD OLED QLED 4K Ultra HD TV Remote Control with YouTube & Netflix Function [ Compatible for Sony Tv Remote Control ] Accessories 3.5  
7SEVEN® Suitable Sony Tv Remote Original Bravia for Smart Android Television Compatible for Any Model of LCD LED OLED UHD 4K Universal Sony Remote Control Accessories 3.9

## Evaluation Metrics

```

Content Precision@5: 0.0000
CF Precision@5: 0.0000
Hybrid Precision@5: 0.0000
```

### Insight:

```
=====
Hybrid model combines strengths of CF and content-based approaches.
It performs better in sparse data scenarios.
```

```
=====
Summary: Content focuses on product similarity,
CF focuses on user behavior patterns,
Hybrid balances both for best overall results.
```

```
=====
None
```

**Observation:** All models (Content, CF, Hybrid) show Precision@5 = 0.0 due to extreme sparsity.

Each user has very few interactions ( $\approx 1$ ), and after train-test split, the model cannot recover the exact test item.

This leads to zero overlap between recommended and actual items.

### Conclusion:

- Precision@K is not reliable for this dataset
- Alternative evaluation metrics (coverage, diversity, hit rate) should be considered
- Hybrid model is still valuable as it improves recommendation quality qualitatively

```
In [219... compare_models(user_item_matrix.index[0]) # returning user
```

```

=====
Model Comparison for User: AE22Y3KIS7SE6LI3HE2VS6WWPU4Q,AHWEY02IJ5I5GDWZAHJK6NGYH
FMA,AGYURQ3476BNT4D2O46THXEUY3SA,AFPMBBSBIEX450Q6UCQWPDG55GWLQ,AGWJU3WUQBDQYPSYAJ5
R3AKBLCOA,AEOVUNFCIFV2230536GVW5JHZKOA
=====

```

#### 📦 Content-Based Recommendations

```

product_name cat_lvl3 rating
Sony TV - Remote Compatible for Sony LED Remote Control Works with
h Sony LED TV by Trend Trail Speed tech & Remote hi Remote & REO India only Acces
series 3.8
7SEVEN® Suitable Sony Tv Remote Original Bravia for Smart Android Television Co
mpatible for Any Model of LCD LED OLED UHD 4K Universal Sony Remote Control Acces
series 3.9
LRIPL Co
mpatible Sony Bravia LCD/led Remote Works with Almost All Sony led/LCD tv's Acces
series 3.9
7SEVEN® Compatible Vu Smart Tv Remote Control Suitable for Original 4K Android LE
D Ultra HD UHD Vu Tv Remote with Non Voice Feature without google assistant Acces
series 3.7
Electvision R
emote Control for led Smart tv Compatible with VU Smart Led (Without Voice) Acces
series 3.8

```

#### 👥 Collaborative Filtering Recommendations

Precision: 0.0

```

product_name cat_lvl1 rating
Syncwire LTG to USB Cable for Fast Charging Compatible with Phone
5/ 5C/ 5S/ 6/ 6S/ 7/8/ X/XR/XS Max/ 11/12/ 13 Series and Pad Air/Mini, Pod & Othe
r Devices (1.1 Meter, White) Computers & Accessories 5.0
REDTECH USB-C to Lightning Cable 3.3FT, [Apple MFi Certified] Lightning to Type C
Fast Charging Cord Compatible with iPhone 14/13/13 pro/Max/12/11/X/XS/XR/8, Suppo
rts Power Delivery - White Computers & Accessories 5.0
Amazon Basics Wireless Mouse | 2.4 GHz Connection, 160
0 DPI | Type - C Adapter | Upto 12 Months of Battery Life | Ambidextrous Design |
Suitable for PC/Mac/Laptop Computers & Accessories 5.0
Swiffer Instant Electric Water Heater Faucet Tap Home-Kitchen Instantane
ous Water Heater Tank less for Tap, LED Electric Head Water Heaters Tail Gallon C
omfort(3000W) ((Pack of 1)) Home & Kitchen 4.8
Instant Pot Air Fryer, Vortex 2QT, Touch Control Panel, 360° EvenCrisp™
Technology, Uses 95 % less Oil, 4-in-1 Appliance: Air Fry, Roast, Bake, Reheat
(Vortex 1.97Litre, Black) Home & Kitchen 4.8

```

#### 🔗 Hybrid Recommendations (0.6 CF + 0.4 Content+0.1 popularity)

```

product_name cat_lvl3 rating
7SEVEN® Compatible for Samsung Smart 4K Ultra HD TV Monitor Remote Control Replac
ement of Original Samsung TV Remote for LED OLED UHD QLED and Suitable for 6 7 8
Series Samsung TV with Hot Keys BN59-01259E Accessories 3.6
Sony TV - Remote
Compatible for Sony LED Remote Control Works with Sony LED TV by Trend Trail Spee
d tech & Remote hi Remote & REO India only Accessories 3.8
Cryo™ Universal Remote Compatible with Tata Sky Universal HD & SD Set top Box (A
lso Works with All TV) Accessories 3.8

```

LOHAYA LCD/LED Remote Compatible for Sony Bravia Smart L  
CD LED UHD OLED QLED 4K Ultra HD TV Remote Control with YouTube & Netflix Function [ Compatible for Sony Tv Remote Control ] Accessories 3.5  
7SEVEN® Suitable Sony Tv Remote  
e Original Bravia for Smart Android Television Compatible for Any Model of LCD LED  
D OLED UHD 4K Universal Sony Remote Control Accessories 3.9

## Evaluation Metrics

-----  
Content Precision@5: 0.0000  
CF Precision@5: 0.0000  
Hybrid Precision@5: 0.0000

### =====

#### Insight:

Hybrid model combines strengths of CF and content-based approaches.  
It performs better in sparse data scenarios.

=====

Summary: Content focuses on product similarity,  
CF focuses on user behavior patterns,  
Hybrid balances both for best overall results.

=====

## Compare Models

Model | Cold Start User | Cold Start Product | Personalization

Content |  |  | Low

CF |  |  | High

Hybrid |  |  | High

## Cold Start Testing

In [220...

```
— Cold-Start Product Recommendation Function —

def recommend_for_new_product(product_description, top_n=5):
 """
 Recommends similar products for a brand new product
 that does not exist in the dataset yet.
 Uses TF-IDF vector of the description to find nearest matches.
 """

 print("=" * 60)
 print("Cold-Start Product Recommendation")
 print(f"New Product Description: '{product_description}'")
 print("=" * 60)

 # Vectorize the new product description using existing TF-IDF vocabulary
 new_vec = tfidf.transform([product_description])

 # Compute similarity against all existing products
 sim_scores = cosine_similarity(new_vec, tfidf_matrix).flatten()

 # Get top N indices sorted by similarity (descending)
 top_idx = sim_scores.argsort()[::-1][:top_n]
```

```

Fetch matching products
results = df_products.iloc[top_idx][
 ['product_name', 'cat_lvl3', 'discounted_price', 'rating']
].copy()

results['similarity_score'] = sim_scores[top_idx].round(4)

print(f"\nTop {top_n} Similar Products Found:")
print("-" * 40)
print(results.to_string(index=False))

Diversity check
unique_cats = results['cat_lvl3'].nunique()
print(f"\nDiversity: {unique_cats} unique sub-categories across {top_n} results")
print("=" * 60)

return results

```

*# — Test with different new product descriptions* 

---

*# Test 1 – Electronics*

```
recommend_for_new_product("smart home security camera wifi night vision")
```

*# Test 2 – Audio*

```
recommend_for_new_product("wireless bluetooth earphones noise cancelling sports")
```

*# Test 3 – Kitchen*

```
recommend_for_new_product("electric pressure cooker stainless steel 5 litre")
```

```
=====
Cold-Start Product Recommendation
New Product Description: 'smart home security camera wifi night vision'
=====
```

Top 5 Similar Products Found:

-----

| product_name                                                                                                                                                                                                            | cat_lvl3 | discounted_price | rating | similarity_score |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|--------|------------------|
| Imou 360° 1080P Full HD Security Camera, Human Detection, Motion Tracking, 2-Way Audio, Night Vision, Dome Camera with WiFi & Ethernet Connection, Alexa Google Assistant, Up to 256GB SD Card Support SecurityCameras  |          | 2299.0           | 4.1    | 0.3996           |
| CP PLUS 2MP Full HD Smart Wi-fi CCTV Security Camera   360° with Pan Tilt   Two Way Talk   Cloud Monitor   Motion Detect   Night Vision   Supports SD Card (Up to 128 GB)   Alexa & Ok Google   CP-E21A SecurityCameras |          | 1999.0           | 3.8    | 0.3772           |
| TP-Link Tapo 360° 2MP 1080p Full HD Pan/Tilt Home Security Wi-Fi Smart Camera  Alexa Enabled  2-Way Audio  Night Vision  Motion Detection  Sound and Light Alarm  Indoor CCTV (Tapo C200) White SecurityCameras         |          | 2499.0           | 4.2    | 0.2722           |
| Qubo Smart Cam 360 from Hero Group   Made in India   2MP 1080p Full HD   CCTV Wi-Fi Camera   360 Degree Coverage  Two Way Talk   Mobile App Connectivity   Night Vision   Cloud & SD Card Recording SecurityCameras     |          | 2490.0           | 4.1    | 0.2646           |
| MI 360° Home Security Wireless Camera 2K Pro with Bluetooth Gateway BLE 4.2   Dual Band Wi-fi Connection   3 Million 1296p  Full Color in Low-Light   AI Human Detection, White SecurityCameras                         |          | 4499.0           | 4.3    | 0.2428           |

Diversity: 1 unique sub-categories across 5 results

```
=====
Cold-Start Product Recommendation
New Product Description: 'wireless bluetooth earphones noise cancelling sports'
=====
```

Top 5 Similar Products Found:

-----

| product_name                                                                                                                                                                                                       | cat_lvl3 | discounted_price | rating | similarity_score |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|--------|------------------|
| JBL Go 2, Wireless Portable Bluetooth Speaker with Mic, JBL Signature Sound, Vibrant Color Options with IPX7 Waterproof & AUX (Blue) Speakers                                                                      |          | 1999.0           | 4.3    | 0.2182           |
| PTron Tangentbeat in-Ear Bluetooth 5.0 Wireless Headphones with Mic, Enhanced Bass, 10mm Drivers, Clear Calls, Snug-Fit, Fast Charging, Magnetic Buds, Voice Assistant & IPX4 Wireless Neckband (Black) Headphones |          | 599.0            | 3.9    | 0.2080           |
| PTron Tangent Lite Bluetooth 5.0 Earphones with Mic, Hi-Fi Stereo Sound Neckband, 8Hrs Playtime, Lightweight Snug-fit in-Ear Headphones, IPX4 Water Resistant, Fast Charge & Voice Assistant (Black) Headphones    |          | 599.0            | 3.5    | 0.1915           |
| Redgear Cosmo 7,1 USB Gaming Wired Over Ear Headphones With Mic With Virtual Surround Sound, 50mm Driver, RGB LEDs & Remote Control (Black) PC Gaming Peripherals                                                  |          | 1990.0           | 4.3    | 0.1914           |
| PTron Boom Ultima 4D Dual Driver, in-Ear Gaming Wired Headphones with in-line Mic, Volume Control & Passive Noise Cancelling Boom 3 Earphones - (Dark Blue) Headphones                                             |          | 299.0            |        |                  |

3.6 0.1892

Diversity: 3 unique sub-categories across 5 results

=====

Cold-Start Product Recommendation

New Product Description: 'electric pressure cooker stainless steel 5 litre'

=====

Top 5 Similar Products Found:

-----

| product_name                                                                                                                                                                                                                    | cat_lvl3 | discounted_price | rating | similarity_score |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|--------|------------------|
| iBELL MPK120L Premium Stainless Steel Multi Purpose Kettle/Cooker with Inner Pot 1.2 Litre (Silver) SmallKitchenAppliances                                                                                                      |          | 1456.0           | 4.1    | 0.4159           |
| iBELL Castor CTEK15L Premium 1.5 Litre Stainless Steel Electric Kettle,1500W Auto Cut-Off Feature,Silver SmallKitchenAppliances                                                                                                 |          | 664.0            | 4.1    | 0.3656           |
| Hindware Atlantic Compacto 3 Litre Instant water heater with Stainless Steel Tank, Robust Construction, Pressure Relief Valve And I-thermostat Feature (White And Grey) WaterHeaters & Geysers                                  |          | 2399.0           | 4.1    | 0.3522           |
| KENT Smart Multi Cooker Cum Kettle 1.2 Liter 800 Watts, Electric Cooker with Steamer & Boiler for Idlis, Instant Noodles, Momos, Eggs, & Steam Vegetables, Inner Stainless Steel & Cool Touch Outer Body SmallKitchenAppliances |          | 1599.0           | 3.7    | 0.3047           |
| Pigeon by Stovekraft Amaze Plus Electric Kettle (14313) with Stainless Steel Body, 1.8 litre, used for boiling Water, making tea and coffee, instant noodles, soup etc. 1500 Watt (Silver) SmallKitchenAppliances               |          | 699.0            | 3.9    | 0.2886           |

Diversity: 2 unique sub-categories across 5 results

=====

Out[220...

|             | product_name                                      | cat_lvl3               | discounted_price | rating | similarity_score |
|-------------|---------------------------------------------------|------------------------|------------------|--------|------------------|
| <b>1288</b> | iBELL MPK120L Premium Stainless Steel Multi Pu... | SmallKitchenAppliances | 1456.0           | 4.1    | 0.4159           |
| <b>1148</b> | iBELL Castor CTEK15L Premium 1.5 Litre Stainle... | SmallKitchenAppliances | 664.0            | 4.1    | 0.3656           |
| <b>1034</b> | Hindware Atlantic Compacto 3 Litre Instant wat... | WaterHeaters & Geysers | 2399.0           | 4.1    | 0.3522           |
| <b>1020</b> | KENT Smart Multi Cooker Cum Kettle 1.2 Liter 8... | SmallKitchenAppliances | 1599.0           | 3.7    | 0.3047           |
| <b>1041</b> | Pigeon by Stovekraft Amaze Plus Electric Kettl... | SmallKitchenAppliances | 699.0            | 3.9    | 0.2886           |

### Cold Start User

In [221...

```
hybrid_recommend("new_user_123", reference_product_id=df_products['product_id']).
```

Cold-start user → using content-only

Out[221...

|            | product_id | product_name                                      | cat_lvl3             | discounted_price | rating |
|------------|------------|---------------------------------------------------|----------------------|------------------|--------|
| <b>220</b> | B07JH1CBGW | Wayona Nylon Braided Usb Syncing And Charging ... | Cables & Accessories | 649.0            | 4.2    |
| <b>42</b>  | B07JW1Y6XV | Wayona Nylon Braided 3A Lightning to USB A Syn... | Cables & Accessories | 399.0            | 4.2    |
| <b>89</b>  | B07JH1C41D | Wayona Nylon Braided (2 Pack) Lightning Fast U... | Cables & Accessories | 649.0            | 4.2    |
| <b>80</b>  | B07LGT55SJ | Wayona Usb Nylon Braided Data Sync And Chargin... | Cables & Accessories | 399.0            | 4.2    |
| <b>166</b> | B07JPJZ2H  | Wayona Nylon Braided Lightning USB Data Sync &... | Cables & Accessories | 399.0            | 4.2    |

### Cold Start Product

Use new product as reference → content similarity still works.

## All Model Evaluations

In [224...

```
def evaluate_all_models(users, k=5):

 results = []

 all_cf_items = set()
 all_content_items = set()
 all_hybrid_items = set()

 def precision(actual, pred):
 if len(pred) == 0:
 return 0
 return len(set(actual) & set(pred)) / len(pred)

 def diversity_score(recs):
 if len(recs) == 0:
 return 0
 cats = df_products[df_products['product_id'].isin(recs)]['cat_lv13']
 return cats.nunique() / len(recs)

 for user in users:

 actual = test[test['user_id'] == user]['product_id'].tolist()

 # — CF —
 cf_recgs = recommend_products_cf(user, top_n=k)
 cf_ids = list(cf_recgs.index) if hasattr(cf_recgs, "index") else cf_recgs

 # — Content —
 if user in user_item_matrix.index:
 user Rated = user_item_matrix.loc[user]
 user Rated = user Rated[user Rated > 0]

 if len(user Rated) > 0:
 ref = user Rated.idxmax()
 content_recgs = recommend_similar_products_v2(
 ref, product_indices, cosine_sim, df_products, k
)
 content_ids = content_recgs['product_id'].tolist()
 else:
 content_ids = []
 else:
 content_ids = []

 # — Hybrid —
 hybrid_recgs = hybrid_recommend(user, top_n=k)
 hybrid_ids = hybrid_recgs['product_id'].tolist()

 # Collect for coverage
 all_cf_items.update(cf_ids)
 all_content_items.update(content_ids)
 all_hybrid_items.update(hybrid_ids)

 results.append({
 "CF_precision": precision(actual, cf_ids),
 "Content_precision": precision(actual, content_ids),
 "Hybrid_precision": precision(actual, hybrid_ids),
```

```

 "CF_diversity": diversity_score(cf_ids),
 "Content_diversity": diversity_score(content_ids),
 "Hybrid_diversity": diversity_score(hybrid_ids),
 })

df_results = pd.DataFrame(results)

— Average Metrics —
avg_metrics = df_results.mean()

total_products = df_products['product_id'].nunique()

coverage = {
 "CF": len(all_cf_items) / total_products,
 "Content": len(all_content_items) / total_products,
 "Hybrid": len(all_hybrid_items) / total_products
}

— Final Table —
summary = pd.DataFrame({
 "Precision@5": [
 avg_metrics["CF_precision"],
 avg_metrics["Content_precision"],
 avg_metrics["Hybrid_precision"]
],
 "Coverage": [
 coverage["CF"],
 coverage["Content"],
 coverage["Hybrid"]
],
 "Diversity": [
 avg_metrics["CF_diversity"],
 avg_metrics["Content_diversity"],
 avg_metrics["Hybrid_diversity"]
]
}, index=["CF", "Content", "Hybrid"])

print("\n 🏁 Final Model Comparison")
print("="*60)
print(summary.round(4))

return summary

```

## Metric | Meaning

Precision | Accuracy

Coverage | Variety of products

Diversity | Variety of categories

In [225...

```

users = user_item_matrix.index[:100]

evaluate_all_models(users)

```

## Final Model Comparison

|         | Precision@5 | Coverage | Diversity |
|---------|-------------|----------|-----------|
| CF      | 0.000       | 0.0037   | 0.800     |
| Content | 0.002       | 0.3015   | 0.274     |
| Hybrid  | 0.000       | 0.2526   | 0.324     |

Out[225...

|         | Precision@5 | Coverage | Diversity |
|---------|-------------|----------|-----------|
| CF      | 0.000       | 0.003704 | 0.800     |
| Content | 0.002       | 0.301481 | 0.274     |
| Hybrid  | 0.000       | 0.252593 | 0.324     |

### Metric limitation:

Precision@K is low due to sparsity; therefore, coverage and diversity are also considered.

**observation** : Hybrid model achieves better overall performance by combining collaborative filtering and content-based approaches.

- CF struggles due to sparsity (low precision, low coverage)
- Content improves diversity and coverage
- Hybrid balances both, giving better overall recommendations

### Evaluation Statement

The hybrid recommender integrates collaborative filtering (60%) and content similarity (40%) to balance personalization with cold-start robustness. Compared to standalone models, the hybrid system demonstrated improved recommendation stability, better coverage, and enhanced relevance for both new and returning users.

## Section F: Business Strategy & Deployment

Goal Connect your analytical and modeling work to real-world business use cases. Hints Decide which recommendation approaches are better suited for new users versus returning users. Think about strategies for recommending products that have very little or no user feedback. Outline how this system could be deployed, monitored, and updated in a production environment. Success Measurement Identify key metrics that reflect user engagement and business impact. Explain why these metrics are important for evaluating a recommendation system.

### 1. Recommendation Strategy by User Type

#### A. New Users (Cold-Start Users)

Problem:

No interaction history

Collaborative filtering fails

Best Approach:

- ✓ Content-Based Filtering
- ✓ Popularity-Based Ranking
- ✓ Trending Items

Strategy:

- Use category-level browsing signals
- Recommend high-rated, high-engagement products
- Personalize gradually as interactions increase

Why It Works:

Content-based models rely on metadata rather than historical behavior.

## **B. Returning Users**

Problem:

- Need strong personalization
- Avoid repetitive suggestions

Best Approach:

- ✓ Hybrid Model (0.6 CF + 0.4 Content)

Why Hybrid?

- CF captures user behavior patterns
- Content ensures relevance and diversity
- Reduces overspecialization

Business Impact:

- Higher CTR
- Higher repeat purchase rate
- Better user retention

## **C. New Products (No Ratings / No Reviews)**

Problem:

- CF cannot recommend new items
- Risk of low visibility

Strategy:

- ✓ Content similarity matching
- ✓ Temporary popularity boost
- ✓ "New Arrival" ranking factor

Why?

Prevents product launch bias and improves discovery.

## 2. Recommending Products with No Feedback

For products with little or no ratings:

Use:

1. Category similarity
2. Text similarity (TF-IDF / embeddings)
3. Seller credibility score
4. Initial engagement metrics (clicks, impressions)

Business Reasoning:

Avoid hiding potentially high-quality products due to lack of early reviews.

## 3. Production Deployment Architecture

A real-world deployment could look like this:

 Architecture Overview

Data Layer

- Store raw data in AWS S3 or Cloud Storage
- Use ETL pipelines (Airflow / AWS Glue)

Processing Layer

- Spark or Pandas for preprocessing
- Scheduled batch jobs for model retraining

Model Training

- Python (Scikit-learn)
- Offline training every 24 hours

## Model Serving

- FastAPI or Flask REST API
- Docker containerization
- Deployed on AWS EC2 / Kubernetes

## Real-Time Layer

- Redis cache for fast recommendation retrieval
- Real-time user session tracking

## Model Update Strategy

Daily retraining for CF model

Weekly full model refresh

Real-time content-based recommendations

Periodic hyperparameter tuning

## 4. Monitoring & Maintenance

A deployed recommendation system must be monitored continuously.

Monitor:

- Model drift (Evidently AI or Population Stability Index (PSI) for model drift)
- User engagement drop
- Cold-start performance
- Latency
- Error rates

Use:

- CloudWatch / Prometheus
- A/B testing framework

## 5 Success Measurement (KPIs)

Recommendation systems are evaluated based on business impact, not just model accuracy.

1. Revenue Per User (RPU)

CTR= Impressions Clicks

Why Important? Measures recommendation attractiveness.

## 2. Conversion Rate

Conversion = Clicks / Purchases

Why Important? Measures revenue impact.

## 💰 3. Revenue Per User (RPU)

Why? Shows financial contribution of recommendations.

## 🔄 4. Repeat Purchase Rate

Why? Measures long-term engagement.

## 🎯 5. Recommendation Coverage

% of products that get recommended.

Why? Prevents popularity bias.

## 🔄 6. Diversity Score

Ensures variety in recommendations.

Why? Improves discovery and reduces user fatigue.

## 6. Why These Metrics Matter

Metric | Business Value

CTR | Engagement

Conversion | Revenue

RPU | Profitability

Retention | Long-term growth

Coverage | Fairness

Diversity | User satisfaction

A model with high accuracy but low CTR is not business-effective.

## Final Strategic Conclusion

The hybrid recommendation system bridges analytical modeling and business execution by combining personalization, cold-start robustness, and real-world constraints. When deployed with proper monitoring and KPI tracking, it can significantly improve user engagement, conversion rates, and overall revenue impact.